

OutDoor Roomba

DESIGN DOCUMENT

Team Number:

SDDEC25-16

Advisor:

Dr. Diane Rover

Team Members/Roles:

Mitchell Owen: Electrical Engineer, Tester

Brodie Bates: Embedded Engineer, Designer

Om Shukla: Frontend Engineering

Daniel Vergara Pinilla: Machine Learning Engineer, Backend Engineering

Sudipta Halder: Designer, Embedded Engineer

Team Email:

sddec25-16@iastate.edu

Team Website:

<https://sddec25-16.sd.ece.iastate.edu/>

Revised:

12/9/2025

Executive Summary

The Outdoor Rumble project, named Rumble, aims to develop an autonomous robot capable of detecting and collecting cans and bottles in outdoor environments such as parks, sidewalks, and residential areas. With over 50 billion pieces of litter polluting waterways and roads in the U.S. alone, our team recognized the urgent need for a more scalable and efficient solution than manual trash pickup. Rumble is designed to autonomously navigate complex outdoor terrains, identify common litter items like cans and plastic bottles using machine learning, and safely transport them to onboard storage.

Equipped with camera, lidar, ultrasonic distance sensors and other sensors, Rumble can detect obstacles such as pedestrians, vehicles, and structures, enabling safe navigation through public spaces. The system leverages high-level navigation and integrates cans and bottles detection models for precision targeting. When the battery runs low, Rumble autonomously returns to a charging dock without human intervention. Additionally, it maintains periodic communication with a remote server for data logging and potential future updates.

Our solution combines hardware engineering, embedded systems, machine learning, and systems integration to address the widespread issue of outdoor litter. While our current focus is on collecting lightweight debris like cans and bottles, the long-term vision is to scale this technology into a fleet of eco-friendly robots that contribute to cleaner, safer communities. We believe that Rumble is more than just a class project—it's a step toward sustainable urban cleanup through automation.

Learning Summary

Development Standards & Practices Used

- IEEE Standard Glossary of Computer Hardware Terminology (610.10)
- ISO/IE/IEEE International Standard – Software and Systems Engineering
- IEEE 2846-2022 – IEEE Standard for Assumptions in Safety-Related models for Automated Driving Systems
- IEEE SA - P3474 Standard for Human Intentions and Artificial Intelligence Alignment in Autonomous Driving Agent
- IEEE 7004-2021 – Ethical Requirements Testing for Autonomous Robotic Devices

Summary of Requirements

1. Functional Requirements

1.1 Cleaning and Collection

FR-1: The system shall detect and collectible objects, including cans and bottles.

FR-2: The system shall collect and transport cans and bottles.

FR-3: The system shall clean the designated operational area.

1.2 Navigation

FR-4: The system shall navigate around non-negotiable obstacles without disturbing them.

1.3 Localization and Connectivity

FR-5: The system shall receive obstacle information from the server when the obstacle is outside the robot's hardware detection range.

FR-6: The system shall support Unified Endpoint Management (UEM) to allow multiple robots to connect and share environmental data.

1.4 Maintenance and Access

FR-7: The system shall provide tool-assisted access to the trash compartment for cleaning and maintenance purposes.

2. Non-Functional Requirements

2.1 Performance

NFR-1: The system shall operate for a minimum of 1 hour on a single battery charge.

2.2 Reliability

NFR-3: The system shall avoid causing damage to property under all normal operating conditions.

2.3 Maintainability

NFR-4: The system shall be designed to allow low-cost repairs.

NFR-5: The system shall use modular components to simplify maintenance.

2.4 Safety

NFR-6: The system shall maintain safe distances from people, animals, and obstacles during operation.

NFR-7: The system shall comply with operational safety requirements to mitigate risks, as insurance does not cover damage related to autonomous robots.

3. Constraints

3.1 Budget and Cost

C-1: The system shall have a total material cost not exceeding \$500, as required by ETG funding.

3.2 Environmental and Operational

C-2: The system shall not operate in the presence of wild animals or large crowds.

3.3 Infrastructure

C-3: The server infrastructure shall support simultaneous connections from multiple robots.

3.4 Operational Risk

C-4: The system design shall minimize the likelihood of property damage resulting from inadequate insurance coverage for autonomous systems.

Applicable Courses from Iowa State University Curriculum

List all Iowa State University courses whose contents were applicable to your project.

- CPRE 2810- Digital Logic
- CPRE 2880 - Embedded Systems Platform Movement and Code Optimization
- CPRE 4880 - Embedded Systems Design
- COMS 3090 - Software Development Practices
- COMS 3110 - Introduction to the Design and Analysis of Algorithms
- COMS 474 - Intro machine learning

New Skills/Knowledge acquired that was not taught in courses

This is the first major project that we had to work together while all having different degrees. This had a lot of pros and cons to do. We had to teach each other things from each degree to better understand our project as a whole. We also had to learn how to use a Raspberry Pi, because some of us have some familiarity with it, but others never had to work with it before. Some of us were really familiar with designing hardware, but never had the same kind of freedoms we have now. Most times we are given a lab that will walk us through the steps, but this project you had to find your own path from start to finish which was a challenge at first.

Table of Contents

1. Introduction	11
1.1. Problem Statement	11
1.2. Intended Users	11
Public Places	11
Event planners	12
The Property Owner	12
2. Requirements, Constraints, And Standards	13
1.3. Requirements & Constraints	13
1.Functional Requirements	13
1.1 Cleaning and Collection	13
1.2 Navigation	13
1.3 Localization and Connectivity	13
1.4 Maintenance and Access	14
2. Non-Functional Requirements	14
2.1 Performance	14
2.2 Reliability	14
2.3 Maintainability	14
2.4 Safety	14
3. Constraints	15
3.1 Budget and Cost	15
3.2 Environmental and Operational	15
3.3 Infrastructure	15
3.4 Operational Risk	15
1.4. Engineering Standards	15
IEEE SA - 1872.2-2021 IEEE Standard for Autonomous Robotics (AuR) Ontology	15
IEEE SA - P3474 Standard for Human Intentions and Artificial Intelligence Alignment in Autonomous Driving Agent	16
IEEE SA - IEEE 1515-2000 IEEE Recommended Practice for Electronic Power Subsystems: Parameter Definitions, Test Conditions, and Test Methods	16
3. Project Plan	17
3.1 Project Management/Tracking Procedures	17
3.2 Task Decomposition	17
3.3 Project Proposed Milestones, Metrics, and Evaluation Criteria	18
3.4 Project Timeline/Schedule	19
3.5 Risks and Risk Management/Mitigation	21
3.6 Personnel Effort Requirements	23
3.7 Other Resource Requirements	24
4. Design	25
4.1 Design Context	25
4.1.1 Broader Context	25

4.1.2 Prior Work/Solutions	26
4.1.3 Technical Complexity	27
4.2 Design Exploration	27
4.2.1 Design Decisions	27
4.2.2 Ideation	27
4.2.3 Decision-Making and Trade-Off	28
4.3 Proposed Design	30
4.3.1 Overview	30
4.3.2 Detailed Design and Visual(s)	31
4.3.2.1 Embedded System	31
4.3.2.1 Components Layout	31
4.3.2.2 Components	31
4.3.2.2.1 PING Ultrasonic Sensor	31
4.3.2.2.2 AS5600-Magnetic-Encoder-Magnetic	32
4.3.2.2.3 TCA9548A I2C IIC Multiplexer Breakout Board Module	32
4.3.2.2.4 KY-040 Rotary Encoder Module	32
4.3.2.2.5 D200 LiDAR Kit	33
4.3.2.2.6 L298n Motor Driver	33
4.3.2.2.7 Samsung 35E 18650 3500mAh 8A Battery	34
4.3.2.2.8 3S 12V 40A Lithium Battery Board PCB Board	34
4.3.2.2.9 12V to 6V Step Down Converter	34
4.3.2.2.10 DC 12V 200RPM Gear Motor High Torque	35
4.3.2.2.11 TCA9548A I2C IIC Multiplexer Breakout Board Module	35
4.3.2.2 Schematics	36
4.3.3 Functionality	40
4.3.4 Areas of Concern and Development	41
4.4 Technology Considerations	41
5. Testing	42
5.1 Unit Testing	42
5.2 Interface Testing	43
5.3 Integration Testing	43
5.4 System Testing	43
5.5 Regression Testing	44
5.6 Acceptance Testing	44
5.7 User Testing	44
5.8 Results	44
6. Implementation	46
6.Embedded System	46
6.Machine Learning	46
6. CAD	46
6.Server	47

6.Frontend	48
6. Design Analysis	51
7. Ethics and Professional Responsibility	52
7.1 Areas of Professional Responsibility/Codes of Ethics	52
7.2 Four Principles	53
7.3 Virtues	56
8. Closing Material	59
8.1 Conclusion	59
9. References	59
10. Appendices	60
10.1 Appendix: ssd_mobilenet_v2_coco_quant_postprocess_edgetpu.tflite	60
10.2 Appendix: ssd_mobilenet_v2_coco_quant_postprocess.tflite	61
11 Team	64
11.6 Team Contract	65
Team Procedures	65
Participation Expectations	66
Leadership	66
Collaboration and Inclusion	67
Goal-Setting, Planning, and Execution	68
Consequences for Not Adhering to Team Contract	68

Table of Figures

Figure 3.2: Decomposition Diagram – 11
Figure 3.3: Milestone – 13
Figure 3.4.2: Detail Grant Chart – 13
Figure 3.5: Risk Management Chart – 17
Figure 3.6: Personnel Effort Requirements chart - 18
Figure 4.1.1: Broader Context – 19
Figure 4.1.2: Prior Work – 21
Figure 4.2.3: Decision-Making and Trade-Off – 23
Figure 4.3.1 : Overview of Project – 25
figure 4.3.2.1: Components Layout –26
Figure 4.3.2.1: Embedded diagram – 27
Figure 4.3.3: Journey Map –28
Figure 5.7.1: Use of Webot – 33
Figure 6.1: CAD Model – 34
Figure 6.2: Block Diagram Accounts On Server – 35
Figure 6.3: Front-end Application Pages –36
Figure 7.2: Four Principles – 38

1. Introduction

1.1. PROBLEM STATEMENT

Keeping outdoor spaces clean is a challenge that event planners and business owners face. Oftentimes attendees at an event will leave behind their trash and litter. This leads to the event planners or venue owners having to clean up the area which takes significant time, effort and resources.

Our project aims to address this need by creating an autonomous outdoor cleaning robot that will be capable of detecting cans and bottles and cleaning it up all while avoiding damage to property. Our goal is to use this robot to reduce man hours, effort and resources that go into cleaning up after events.

1.2. INTENDED USERS

Public Places

One of the most significant problems in this world is that there's trash everywhere. We often find it in our streets and parks. People are sick of it; they can't go without a day being outside and seeing multiple pieces of trash like beer cans, soda bottles, and other litter on the ground. Sometimes, community volunteers try to keep their neighborhood clean, but sometimes, it's too much to be cleaned in one day, or there isn't a close community in the area in the first place. In some cities, the government helps clean, but there are cities where there's no support at all. In these cases, an automated solution to pick up trash helps. This is where our Outdoor Rumble can help pick up cans and bottles and clean up public places like parks and streets around the clock, but the general public has concerns with this solution. One, it can hit a car or a person and hurt them. Two insurance companies for property and vehicles don't cover damages caused by an automated robot. Our Outdoor Rumble can navigate around people, cars, and other obstacles with its sensors, giving live data on incoming obstacles and its ability to navigate them. The general public will have cleaner streets and a healthier environment, which can help people enjoy their streets and parks without litter.

Event planners

After a big event like a party or a festival, event planners usually have to deal with cleaning up the cans and bottles left behind. All this debris can sometimes be overwhelming work, causing some garbage to be left uncleaned. Not only does this impact the environment and pollute the towns and cities, but it can also result in a heavy fine for the venues and the event planner's company. They can hire more staff to clean up, but this can sometimes be out of budget. So, event planners want to clean up after a party with less effort and without impacting the environment while not damaging the property or hurting people. The Outdoor Rumble can clean up the venues at any time. With its automation logic and sensors, it can avoid obstacles like people, cars, and other objects. The Outdoor Rumble can benefit Event planners by cleaning a large amount of garbage in the venues, making it easier to clean and avoiding any fees or negative impacts on the environment after an event.

The Property Owner

Some property owners often struggle with keeping their properties and pathways free of cans and bottles, especially those large properties (Ex, golf courses). They usually make themselves work or hire workers to keep things neat, making it time-consuming and costly. There are automated grasscutters in the market, but oftentimes there isn't an automotive solution for garbage collection. The property owners need an affordable way to keep their property clean of trash. Our Outdoor Rumble can help property owners keep their place trash-free by taking rounds around the property to pick up cans and bottles at most times of the day. This project will allow property owners to spend less on keeping their property clean. If their property serves as a business (golf courses), it can help with the consumer experience as most consumers would enjoy a cleaner environment.

2. Requirements, Constraints, And Standards

1.3. REQUIREMENTS & CONSTRAINTS

1.Functional Requirements

1.1 *Cleaning and Collection*

FR-1: The system shall detect collectible objects, including(payload) cans and bottles.

Description:

Verification:

FR-2: The system shall collect and transport payload (cans and bottles).

Description:

Verification:

FR-3: The system shall clean the designated operational area.

Description:

Verification:

1.2 *Navigation*

FR-4: The system shall navigate around non-negotiable obstacles without disturbing them.

Description:

Verification:

1.3 *Localization and Connectivity*

FR-5: The system shall receive obstacle information from the server when the obstacle is outside the robot's hardware detection range.

Description:

Verification:

FR-6: The system shall support Unified Endpoint Management (UEM) to allow multiple robots to connect and share environmental data.

Description:

Verification:

1.4 Maintenance and Access

FR-7: The system shall provide tool-assisted access to the trash compartment for cleaning and maintenance purposes.

Description:

Verification:

2. Non-Functional Requirements

2.1 Performance

NFR-1: The system shall operate for a minimum of 1 hour on a single battery charge.

Description:

2.2 Reliability

NFR-3: The system shall avoid causing damage to property under all normal operating conditions.

Description:

2.3 Maintainability

NFR-4: The system shall be designed to allow low-cost repairs.

Description:

NFR-5: The system shall use modular components to simplify maintenance.

Description:

2.4 Safety

NFR-6: The system shall maintain safe distances from people, animals, and obstacles during operation.

Description:

NFR-7: The system shall comply with operational safety requirements to mitigate risks, as insurance does not cover damage related to autonomous robots.

Description:

3. Constraints

3.1 Budget and Cost

C-1: The system shall have a total material cost not exceeding \$500, as required by ETG funding.

Description:

3.2 Environmental and Operational

C-2: The system shall not operate in the presence of wild animals or large crowds.

Description:

3.3 Infrastructure

C-3: The server infrastructure shall support simultaneous connections from multiple robots.

Description:

3.4 Operational Risk

C-4: The system design shall minimize the likelihood of property damage resulting from inadequate insurance coverage for autonomous systems.

Description:

1.4. ENGINEERING STANDARDS

Engineering standards are crucial because they provide a consistent set of guidelines that ensure engineering work's safety, quality, and reliability. They help engineers design by adhering to standards; engineers can minimize errors, reduce risks, and create interoperable and sustainable solutions. These standards also foster trust between engineers, manufacturers, regulators, and the public, ensuring a baseline of performance and safety. Build systems, products, and structures that meet technical specifications and societal needs.

[IEEE SA - 1872.2-2021](#) IEEE Standard for Autonomous Robotics (AuR) Ontology

This standard is for the design of autonomous robots. The Standard provides a unified way to autonomous robotics system architectures in various domains, which include land, air, water, and spacecraft. So designers have a universal standard representing robotics system architectures, which allows other designers to easily understand what hardware and software components are required for the system.

For our project, our robot will be fully autonomous and have a complex system that includes multiple robots, a server, and many sensors. So we need a standard representation that these standards meet. We can follow these standards for our project, so any designer can use this standard to easily read the basic hardware and software components needed to run the robot. Also, we can expand to other opinions since this standard doesn't limit us and allows us to create diagrams and representations since it is applicable to many domains, like land, air, water, so space is the limit. We will document our design using this standard of IEEE AuR Ontology. standard

[IEEE SA - P3474 Standard for Human Intentions and Artificial Intelligence Alignment in Autonomous Driving Agent](#)

This will be good for setting the stage for interaction with humans and our Rumble, since it is automated, and after briefly reading this, we will add expected behavior aligned with human intentions. From the short abstract of what this standard is, it will guide us to figure out what type of interactions this robot will have with humans and how the robot should react to those interactions.

This is relevant to our project because our Rumble will likely be interacting with humans regularly, and we should know what the robot will do and make sure it is safe for people to be near. We could implement this by having our robot detect when there is a human nearby with its camera and artificial intelligence, and based on that, determine what it should do next.

[IEEE SA - IEEE 1515-2000 IEEE Recommended Practice for Electronic Power Subsystems: Parameter Definitions, Test Conditions, and Test Methods](#)

IEEE Recommended Practice for Electronic Power Subsystems: Parameter Definitions, Test Conditions, and Test Methods: This Standard is about how we can ethically use power subsystems. It is mainly about the safety precautions that can be made to make sure our systems will work properly, efficiently, and safe. This would benefit our project because our test conditions will be harsher than most. We will have to ensure that our system can work in different weather conditions, like rain, snow, wind, etc. This is critical for ensuring that our power systems will meet all the safety and efficiency standards, especially since our robot is going to be in different environmental conditions. We must complete additional safety checks in our power systems, following the 1515-2000 standards to ensure stability in extreme conditions.

3. Project Plan

3.1 PROJECT MANAGEMENT/TRACKING PROCEDURES

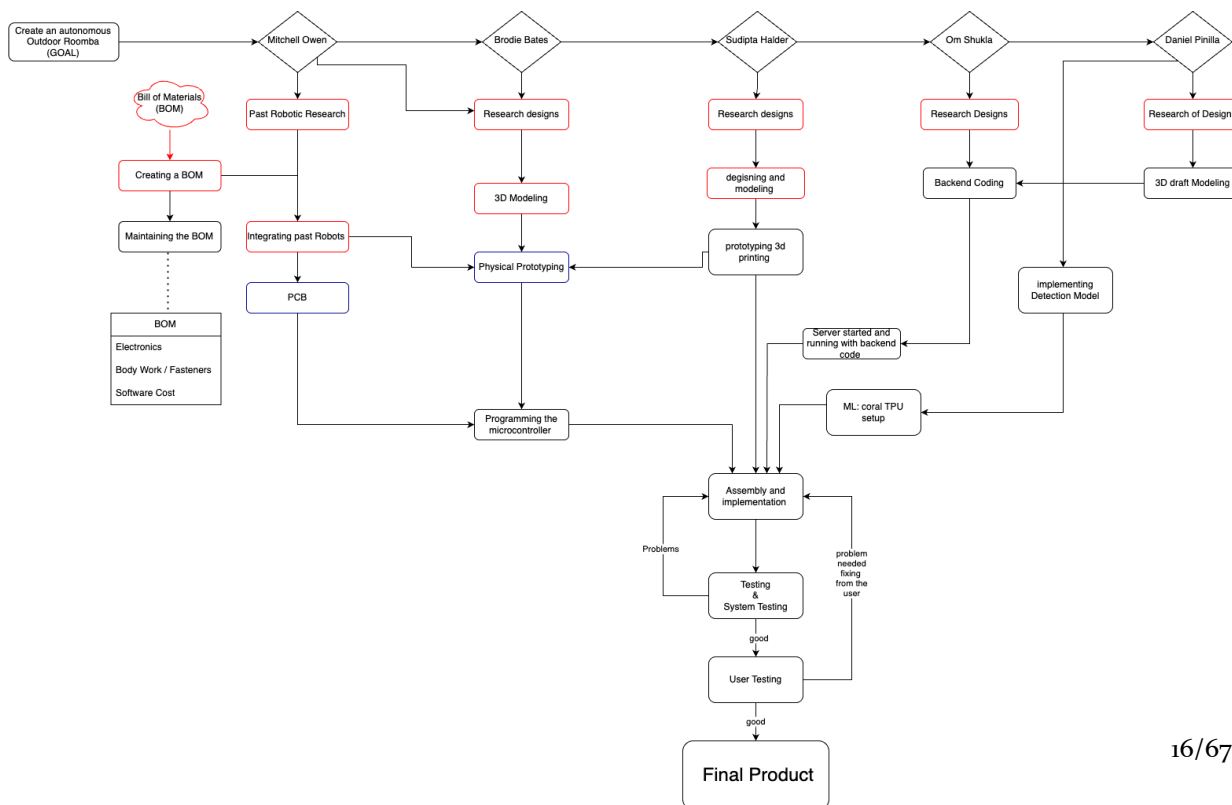
Agile project management style. We use this because agile allows us to modify our requirements quickly. If challenges appear, we can easily change our requirements to allow for developments for the challenge or remove those specific requirements causing the challenges. Our robot will be facing the outdoors. There is a lot of uncertainty about how our robot will perform in various conditions. So Agile is the best and the most forgiving project management style for this project.

We are using Trello to track the overall progress of this project. Trello is simple to add tasks and know what tasks are being worked on and which tasks are done. Most of our code for the robot, server, and user interface will be stored on GitLab. Therefore, most of the progress of the code of the servers, the user interface, and the robot will be tracked on Gitlab's Issue board. Gitlab's Issue board allows tasks (or issues) to work on different branches separated from the main code and files.

3.2 TASK DECOMPOSITION

Robot chassis housing the robot's electronics, and other components that will pick up trash, etc. Next will be designing the robot's embedded system. We will need to figure out what sensors to use and what we can afford with our budget. Find out what motors and actuators we will need. Pick our computing part, like microcontroller computers, etc. Design the PCB. Find out what we will need for a battery that meets the requirements. Next will be building the physical robot according to the design and fitting the electronics into it, and programming the robot's microcontrollers. Next, find or train a machine learning detection of cans, bottles, people, cars, and paths with traffic lights and other road signs. Set up and design the server infrastructure and communication. Lastly, build the user interface for the user. These steps are not really going to be in order. Because there are going to be a lot of changes and we will go back and forth through these tasks, especially the designing of the chassis and the embedded system. We will have to revisit those back-to-back to meet our requirements.

FIGURE 3.2: DECOMPOSITION DIAGRAM



3.3 PROJECT PROPOSED MILESTONES, METRICS, AND EVALUATION CRITERIA

For our project, we have Five Major Milestones that we want to achieve. The first milestone is to create a working physical prototype that can move in our environment, including general areas such as sidewalks and outdoor dining areas in cafes, with no motor stalling. The next milestone is for our trash-picking mechanism to collect plastic and bottles. We can pick up 80% of the time, so out of ten plastic bottles, we should be able to pick up 8 in one try. The next milestone is to have machine learning models that can detect cans and bottles in the real world 60% of the time; this may seem small, but making a machine learning model with high accuracy with its training data can get high results in validation tests (upwards of 90%) but having a model that can detect cans and bottles in the real world we would need a robust dataset which is hard to do, hence the low accuracy rate. The next milestone is to detect objects, static walls, and humans with the combination of our lidar and camera sensors 90% of the time. This milestone is crucial for a proper navigation system, allowing the robot to move effectively. Leading us to the final milestone: the robot can move from point A to point B, avoiding civilians and obstacles to pick up cans and bottles. The robot should be able to reach its desired location within a margin of error of 2 feet or be within a 4-foot radius of the desired location.

Milestones/ Evaluation Criteria	Metrics
Have a physical prototype that moves	Is able to move in outdoor terrain in path
Successfully pick up plastic and aluminum bottles	80% of the time
The trash detection machine learning model via camera image	Detection rate of 60% or greater
The machine learning model for human and car detection and other objects, with a combination of sensors like cameras and lidar for navigation	90% or greater
The robot is able to navigate from point a to point b through obstacles and civilians and pick up cans and bottles	90% of the time, with only a margin of error of two feet $\pm$ away from the desired location

FIGURE 3.3: MILESTONE

3.4 PROJECT TIMELINE/SCHEDULE

Week 0 starts on Feb 17, 2020, and between planned weeks 8 and 9, there's a 2-week gap for team members to focus on the finals and dead week. We will spend the first 6 weeks researching parts, components, and algorithms. But as we research, we will start working on creating servers and modeling since we can make changes according to the research findings. Following week 8, we will begin implementing and developing the machine-learning model. Then, at the beginning of the summer semester, in week 10, we will start working on the embedded side of the project, programming the microcontroller, developing the autonomous features of the robot, and implementing the server with the robot.

Along with creating the robot's embedded side, we will start testing around this time; it should be within the time frame of the second semester. We will also begin user testing when we can have a loop between Testing and user testing. This means we will complete most of the robot in the summer, and the second semester will be like Testing, user testing, and tweaking our robot according to the feedback.

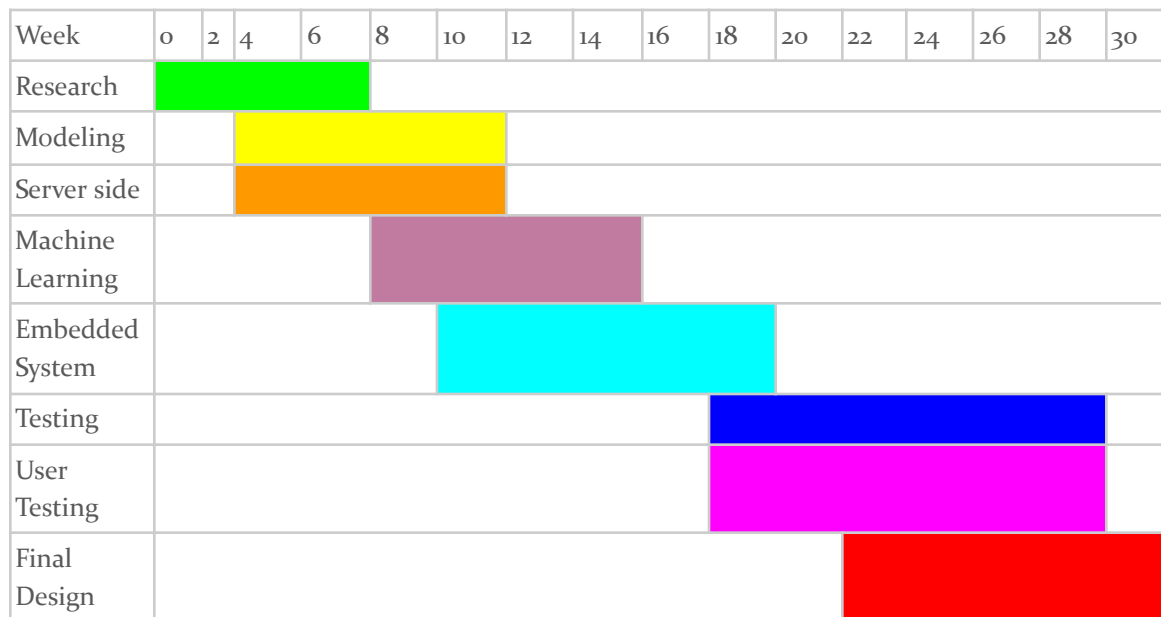


FIGURE 3.4.1: GANTT CHART

Here is a more detailed version of the gantt chart in *Figure 3.4.1* showing all the task we will be trying to get done within this time frame.

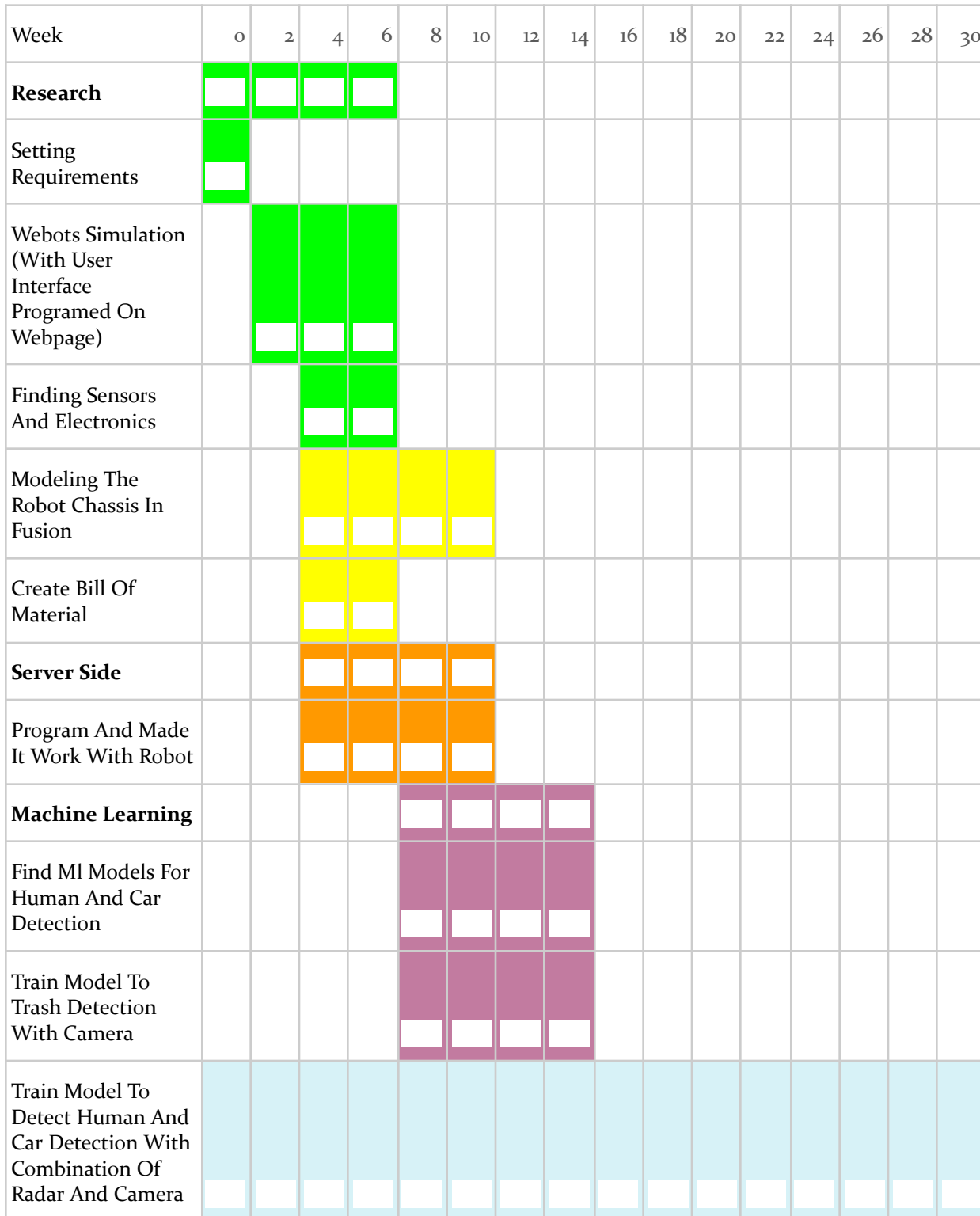




FIGURE 3.4.2: DETAIL GRANT CHART

3.5 RISKS AND RISK MANAGEMENT/MITIGATION

Task	Risk	Chance	Risk Level	How to Fix It
Designing the Robot Chassis	Weak structure	40%	Moderate	CAD simulations, stress testing
	Poor weight distribution	50%	Moderate	Prototype different configurations
	Manufacturing constraints	30%	Low	Consider a premade chassis
Building the Physical Robot	Assembly difficulties	50%	Moderate	Use modular design, precise fabrication
	Durability issues in outdoor conditions	60%	High	Use rugged materials, conduct environmental testing
	Motor and power inefficiencies	40%	Moderate	Test power requirements, use efficient motors
Designing the Embedded System (Sensors)	Sensor integration issues	60%	High	Choose compatible sensors, test connections early
	Signal interference or calibration errors	50%	Moderate	Use shielding and recalibrate regularly
	High power consumption	40%	Moderate	Optimize sensor usage, low-power components
Programming the Embedded System (Sensors)	Data processing lag	60%	High	Optimize code, reduce unnecessary computations
	Software bugs leading to erratic behavior	50%	Moderate	Test modules separately before full integration
	Hardware compatibility issues	40%	Moderate	Use established frameworks and drivers
Machine Learning for Object Detection	Insufficient training data	70%	High	Use pre-trained models, collect real-world data
	Slow processing speed for real-time detection	60%	High	Offload processing to edge/cloud computing
	High power consumption for onboard inference	50%	Moderate	Use efficient power chips
Server Infrastructure & Communication	Latency in cloud communication	50%	Moderate	Use edge computing for real-time processing
	UI usability issues	40%	Moderate	Conduct user testing and feedback loops

FIGURE 3.5: RISK MANAGEMENT CHART

Risk Management/Mitigation TODO

3.6 PERSONNEL EFFORT REQUIREMENTS

Here is a personnel effort requirements chart showing how much estimated time needed to complete a task that is required to complete this project. Requirements definition, Design Research, System Design, Component Selection, Raspberry Pi Setup & Integration, Cad modeling, should be completed by first semester, so by week 10. Then Software development - Navigation, Prototyping, should be complete in the summer semester. Lastly, Testing, Validation, Iteration and Improvements should be done by fall semester.

Task	Duration (Weeks)	Estimated Hours/Week	# of Team Members	Total Person Hours	Notes
Requirements Definition	1	6	4	24	Self-guided requirements definition and brainstorming
Design Research + Competitive Analysis	2	4	4	32	Informs component choices and feasibility
System Design	2	8	4	64	Includes block diagrams, design partitioning
Component Selection	1	6	3	18	Focus on finding sensors, Raspberry Pi, and drive systems
Raspberry Pi Setup & Integration	3	10	2	60	OS Installation, GPIO setup, sensor and motor interfacing
Mechanical CAD Design	3	8	2	48	CAD + stress analysis and enclosure design

Software development - Navigation	4	12	2	96	Path planning and obstacle avoidance
Prototyping	3	10	4	120	Assembly and integration of hardware
Testing + Validation	4	8	3	96	Outdoor environment testing
Iteration & Improvement	2	6	4	48	Updates based on testing results
Final Report & Presentation Prep	2	6	4	48	Drafting, revisiting, and practicing
Buffer / Miscellaneous / Meetings	15	1	4	60	Weekly meetings and coordination

FIGURE 3.6: PERSONNEL EFFORT REQUIREMENTS CHART

The allocation of task durations and workload was informed by a detailed review of the Gantt chart and the goals defined in our project proposal. As a self-proposed initiative, the planning process was entirely internally driven, focusing on requirement definition, design ideation, and feasibility assessment. The 15-week schedule accommodates parallel development tracks for both software and hardware components. Emphasis was placed on high-effort areas such as navigation software and physical prototyping, due to the integration challenges and environmental variability inherent in outdoor robotics. A Raspberry Pi was selected as the central control unit, necessitating dedicated effort for system setup, configuration, and interfacing with sensors and actuators. Team members are expected to contribute between 8 and 12 hours per week during intensive phases, culminating in an estimated total of 714 person-hours. The timeline includes provisions for ongoing meetings, testing, and iterative refinement to ensure the project's alignment with academic expectations and technical success.

3.7 OTHER RESOURCE REQUIREMENTS

Electronics

Materials

Equipment

For this project, we will need the following items: Raspberry Pi, Coral AI, motors, brushless motors, motor controllers, wheels, aluminum, batteries, a thin steel plate, drawer slides, sensors, a 180-degree camera, a lidar sensor. Additionally, we need a server machine that can host the code and database, allowing for communication with the robot.

4. Design

4.1 DESIGN CONTEXT

4.1.1 Broader Context

Area	Description	Examples
Public health, safety, and welfare	The general public will benefit from our robot because there will be less hazards around and keeps everything cleaner and healthier. The robot will be able to detect people, animals, and other objects that they should avoid.	Reducing hazards Reduces property damage reduced personal injuries
Global, cultural, and social	Communities value safety and environmental health which our project will do.	Community Safety Ethical development Encourage technological advancements
Environmental	We will be energy efficient, by being completely electric. The purpose of the robot is also to clean up the environment.	sustainable materials. low carbon emission energy consumption
Economic	The final product must be affordable with affordable components.	Cost-effective solutions cheaper replacement parts

FIGURE 4.1.1: BROADER CONTEXT

4.1.2 Prior Work/Solutions

In the past, many autonomous robots developed to clean an area. There were home robots developed that are widely used. These robots are called Rumbles and have been affordable to mass consumers, but the Rumble is unable to clean large trash bodies like plastic bottles. Then there is a robot made in Japan, S330, which can be autonomous and is designed to clean parks and pick up larger trash like plastic bottles, while navigating civilians. The only problem with the S330 is that it is very costly. Our robot will be similar to S330, but we want it to be affordable, user-friendly, and smaller, similar to the Rumbas used indoors.

Product Services and Design What is the product?	Unique Value Proposition What makes this product unique?	Product Advantages What are the things that provide a leg up?	Product Disadvantages Where might drawbacks exist?	User Pros What do users like about the product?	User Cons What do users NOT like about the product?
 The S330 is an outdoor street cleaner that originated in Hong Kong.	The S330 can switch from an autonomous to a manual mode.	It is Massive to clean up a more significant surface area than most sweepers. It also has a larger battery, which allows it to run for a long time.	It is only a sweeper, meaning it can pick up all the trash but instead clean up dirt. It is a massive room.	The fact that it is autonomous and cleans up a large surface area.	It is costly, takes up a lot of room on the streets, and is very slow.
 The Roomba is a small robot used for daily house cleaning.	The Roomba has bump sensors to determine when an object is in the way. It can also get updates on the status of the Roomba through an app on the user's phone.	The small size and relatively low price point provide easy and convenient access to everyone.	Short battery life. Small storage. Always gets clogged. Has a small cleaning radius. Indoor use only	The small size allows users to keep the Roomba in their house without causing a distraction. Easy to install and use	Replacement parts are expensive. Need Internet to control the app. Always gets clogged.
 The TSM Follow Me is a street vacuum cleaner that can track and follow the operator.	It is a street vacuum cleaner, not a sweeper, and can independently identify and follow the operator.	Requires a human operator so they can pick up anything the robot can't pick up on its own.	It requires a human operator, so it's not entirely independent.	Can navigate complex situations with the help of the human operator.	It can function solely in the presence of a human operator, relying on their guidance and oversight to perform its tasks.

FIGURE 4.1.2: PRIOR WORK

4.1.3 Technical Complexity

The design comprises multiple components utilizing distinct scientific, mathematical, or engineering principles. The movement of our robot will require electrical principles and physics to find the perfect motors that fit our requirements and batteries that last the duration we want. Machine learning is needed to detect cans and bottles. We need a solid understanding of machine learning and theory in computer science. The use of sensors needs an under-embedded system and the skill of decomposition datasheets for sensors and microcontrollers. The server, website, and app need a solid foundation in software development practices. Lastly, developing and optimizing the algorithm used to detect and navigate obstacles via Pathfinder requires a deep understanding of the theory of algorithms.

In the Industry of embedded cleaning machines, there hasn't been an outdoor robot that can, in our size and price, navigate through civilians and other obstacles to pick up plastic waste in urban areas. There has been research about this, but it never turned into a product for the users, nor does it use the server to implement geolocation or have advanced pathfinding algorithms. Since there are not similar products with our design goals it will make our project more challenging.

4.2 DESIGN EXPLORATION

4.2.1 Design Decisions

TODO

We need to make some of the design decisions for the robot's compactor system. That will include multiple sensors and multiple microcontrollers, also we need to decide what languages to use for each component, as well as what electronic components to get to stay within budget. Each of these decisions are important to the project's success because the size of the robot will determine how many cans or bottles it can pick up as well as how long the battery will last.

Several key design decisions were made to ensure our robot meets our requirements. One of these decisions is our robot's size and material. We chose to make the robot smaller so that it will not take up much space however this leads to having less room to store the garbage it picks up. Materials used would need to be water resistant as we planned to have our robot operating outdoors.

Another design decision was which programming languages to choose. For our embedded systems we chose C++ as the main language as it efficiently operates on our microcontrollers. For the Raspberry Pi we use python as it can easily communicate with our backend server and use serial to communicate to the other microcontrollers. For our servers backend we use Java with springboot because

A third major design decision was the batteries. Since our requirements were for the robot to be able to run for three hours, we chose to use a set of three rechargeable batteries, and calculated how much power consumption we should have to determine theoretical runtime.

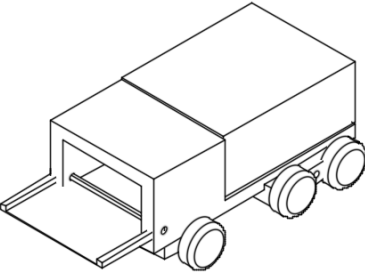
4.2.2 Ideation

One of the design choices the team had to make is the dimensions of the robot. We had a few different designs in mind, all of which were different form factors. To start we had a WALL-E like design. Then we had a truck-like design with tracks, then we decided to make the back removable. Lastly we decided to make it more compact and almost the shape of a cube. Each of these options were decided because we wanted to optimize some aspect of the project. The wider robot would have a better center of gravity which would allow it to not tip over so easily. The truck-like robot would be able to carry more. The WALL-E like design was a starting point for the

project and we started out with tracks on the robot due to it. The back removable design was made in order to easily dispose of what is inside. Finally, the more compact design was made using elements of each of the previous designs but would optimize the cost of the robot overall.

4.2.3 Decision-Making and Trade-Off

We created a Pros and Cons list for the design of our robot and all components we wanted in them. We were stuck between three different designs. The WALL-E design, the Semi Truck design, and the Cubed design. Below you can see our thought process that we went through when deciding what design we wanted and why we decided to pick the cubed design.

Options	Descriptions	Pro's	Con's	Final Decision
WALL-E  PIXAR	Based on the movie WALL-E from the movie pixar. It is more of a cubed robot with his sensors (eyes) extended from his body, also his arms are really long. Triangle Tracked wheels	Good center of gravity. Longer range for trash collection. Smart AI with advanced compactor systems. -high elevated base	Hard to repair -Tracked tires.	We decided against this option for the main purpose of cost. WALL-E is fictional, so they have more range of price when coming up with a robot, but we do not have that luxury.
Semi-Truck 	Based on a semi-truck. It is a cubed front end for the trash to enter and a long rectangular base from a lot more trash.	A lot of capacity for trash Great center of mass. -Lower base and more surface area.	Expensive parts -Extra materials for the longer Trash container. More weight Bigger turn radius.	The main reason we decided against this design is because of two things. The cost and the turn radius. Cost- We can't justify the extra cost when making a robot that's main purpose is to only pick up trash. We need to still keep the cost down so the consumers want to purchase our products. Turn Radius- Just like semi-trucks on the road when they turn they take a wider turn due to its size. We didn't want that especially with some trash potentially being in tighter areas. This could affect accuracy

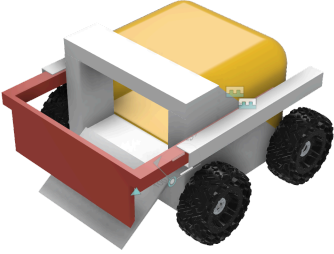
				and cause damage, so we moved
Cubed Robot 	A cube base robot that is more compacted with the sensor built into the top of the robot instead of having any overhang.	Good center of gravity Less material cost and weight More compacted Smaller turn radius	A small capacity of trash	We decided to go with the cubed robot because it hits all of our checkpoints. -low priced -Good center of gravity -Compacted

FIGURE 4.2.3: DECISION-MAKING AND TRADE-OFF

4.3 PROPOSED DESIGN

4.3.1 Overview

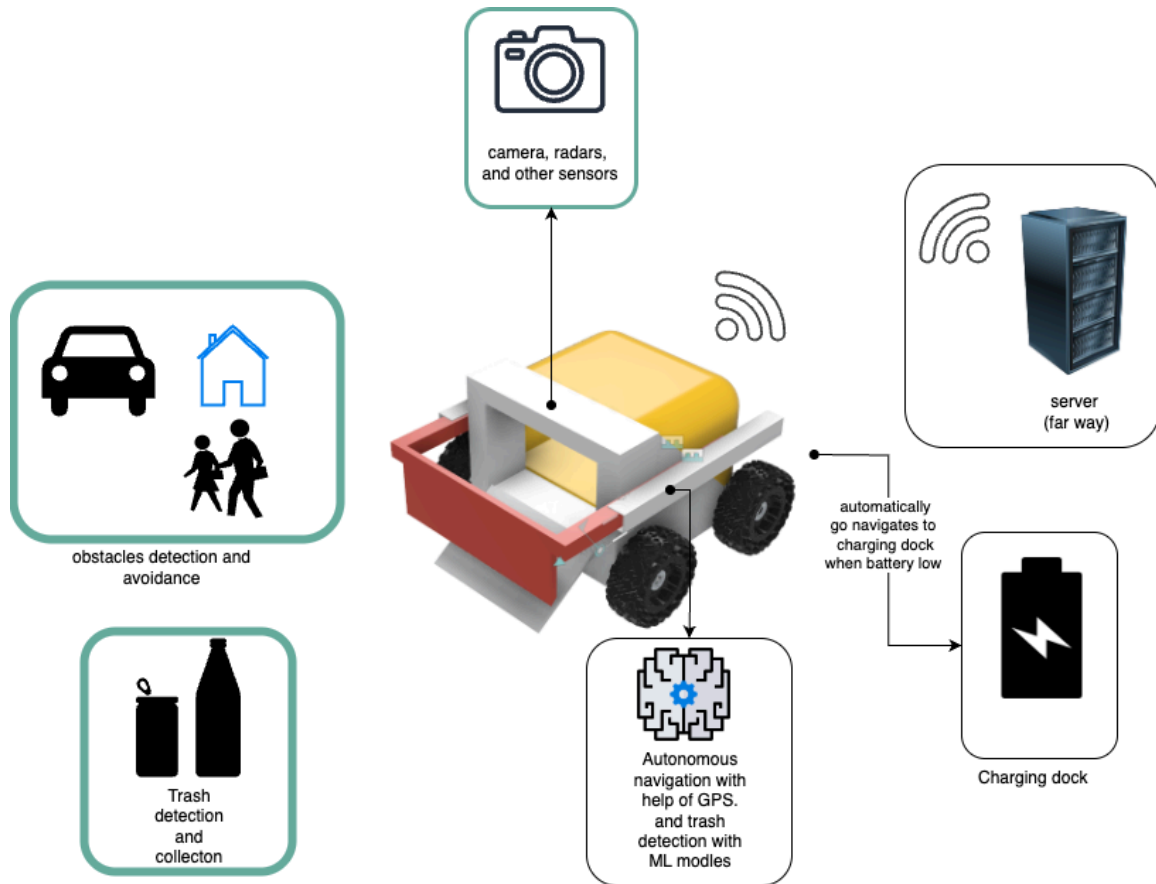


FIGURE 4.3.1 : OVERVIEW OF RUMBLE

Our robot's primary goal is to find and clean up cans and bottles. It will use a bucket mechanism with a "dustpan and brush," where a power shovel will lift the garbage into itself and a bush-like wall in the front to guide the cans or bottles into the trash compartment. The trash compartment will be slanted towards the bottom, making sure the garbage goes all the way to the back, making it easier for the sensor to give an accurate capacity reading. It will use a machine learning model on the camera to find the cans and bottles. The machine learning model on the camera and the lidar and ping ultrasonic sensor will be used to see obstacles in their way to navigate around, so it can detect human cars and vehicles and navigate around them. The robot will also be connected to the server passing information about its current location and objects it's detecting. The robot will also listen to the server for incoming traffic from civilians and other Rumble robots. This way, it can avoid collisions and successfully navigate around it to reach its goal.

4.3.2 Detailed Design and Visual(s)

4.3.2.1 Embedded System

4.3.2.1 Components Layout

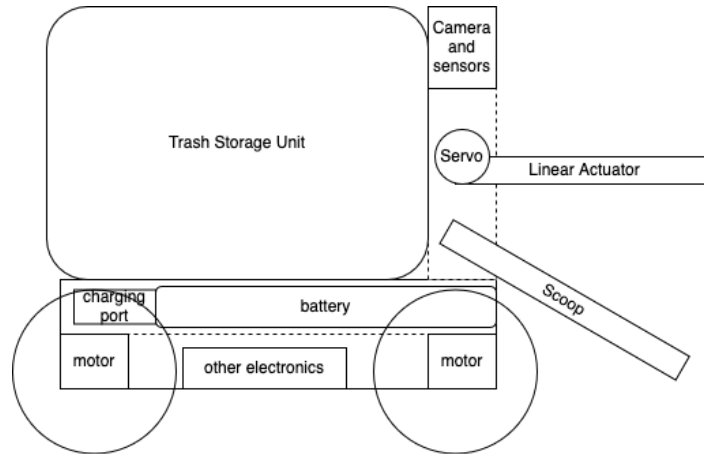


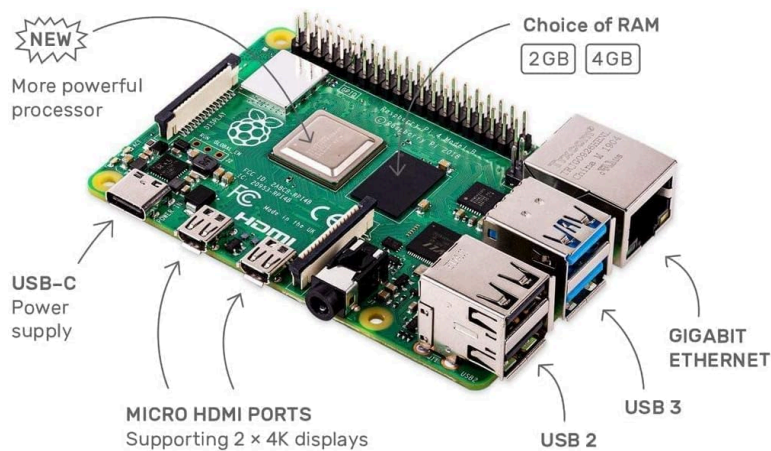
FIGURE 4.3.2.1: COMPONENTS LAYOUT

TODO

This is our high level design for our hardware components. The front of the robot includes the sensors, linear actuator, and scoop which will be our system to pick up trash. The battery and other electronics will be stored inside the chassis in order to protect it from the weather. These electronics will run the entire robot motors, servos, and actuators.

4.3.2.2 Components

TODO



<https://www.amazon.com/Raspberry-Pi-Computer-Suitable-Workstation/dp/Bo899VXM8F>

https://en.wikipedia.org/wiki/Raspberry_Pi_4



<https://www.seeedstudio.com/Coral-USB-Accelerator-p-2899.html?srltid=AfmBOoqQoeL4G6kTL5I6z5FcLwchBonKhgiC7sSJQKBgNlMDucDmWfUvKsc>

4.3.2.2.1 PING Ultrasonic Sensor



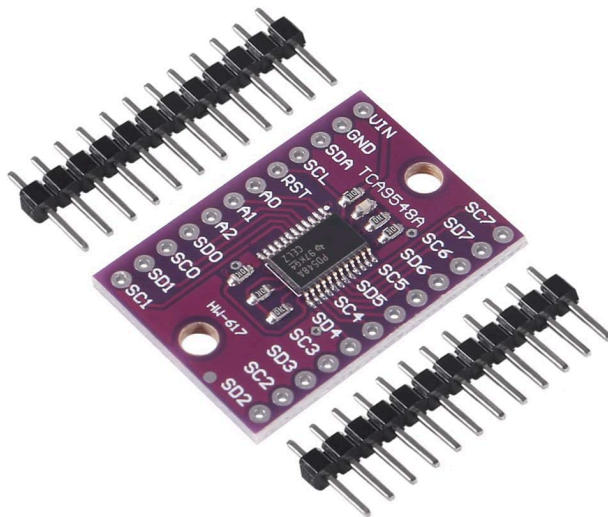
https://www.robotshop.com/products/parallax-ping-ultrasonic-sensor?srltid=AfmBOopnwFzHRHpcq75SSIKtCkIM77HoagevpZ_ImNjcRi82vYMXFQG

4.3.2.2.2 AS5600-Magnetic-Encoder-Magnetic



https://www.alibaba.com/product-detail/AS5600-Magnetic-Encoder-Magnetic-Induction-Angle_1601415773659.html

4.3.2.2.3 TCA9548A I2C IIC Multiplexer Breakout Board Module



https://www.amazon.com/dp/BoB2VSP488?ref=ppx_yozov_dt_b_fed_asin_title&th=1

4.3.2.2.4 KY-040 Rotary Encoder Module



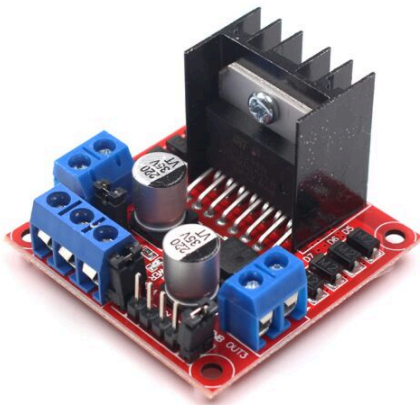
<https://www.parallax.com/product/ky-040-rotary-encoder-module-15x16-5-mm-with-knob/>

4.3.2.2.5 D200 LiDAR Kit



https://www.waveshare.com/wiki/D200_LiDAR_Kit

4.3.2.2.6 L298n Motor Driver



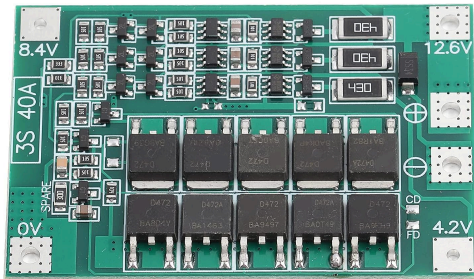
<https://www.makerfabs.com/l298n-motor-driver-board.html?srsltid=AfmBOoqMPSQ-9UAd76WHNUZEZfiKaO-FvmwdLBEEEnSDsfDyUEQIThGTe>

4.3.2.2.7 Samsung 35E 18650 3500mAh 8A Battery



<https://www.18650batterystore.com/products/samsung-35e>

4.3.2.2.8 3S 12V 40A Lithium Battery Board PCB Board



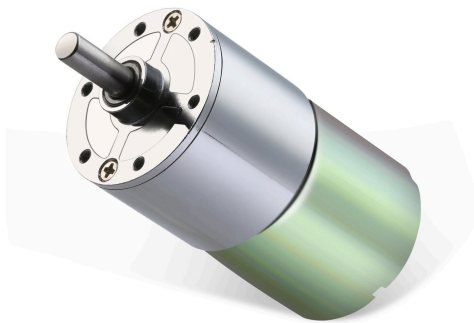
https://www.amazon.com/dp/B07S1SZJWY?ref=ppx_yo2ov_dt_b_fed_asin_title

4.3.2.2.9 12V to 6V Step Down Converter



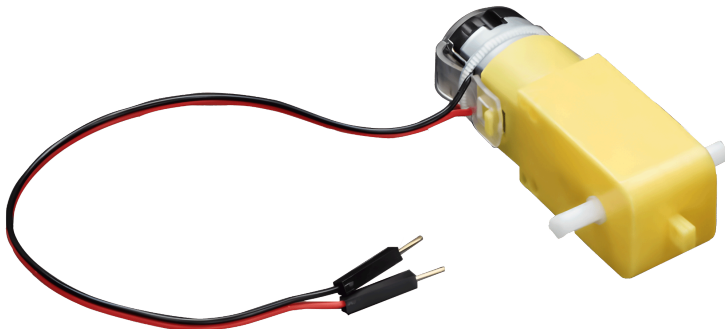
https://www.amazon.com/dp/BoD6Y4XL3N?ref=ppx_yo2ov_dt_b_fed_asin_title&th=1

4.3.2.2.10 DC 12V 200RPM Gear Motor High Torque



https://www.amazon.com/dp/Bo71GTTSV3?ref=ppx_yo2ov_dt_b_fed_asin_title&th=1

4.3.2.2.11 TCA9548A I2C IIC Multiplexer Breakout Board Module



https://docs.sunfounder.com/projects/umsk/en/latest/01_components_basic/34-component_ttmotor.html

https://www.adafruit.com/product/3777?srltid=AfmBOoorRoRkKnruO-QzXC1E8fkWJT9qV_DYlgmrSTUwvjslAl4CxJvO

4.3.2.2 Schematics

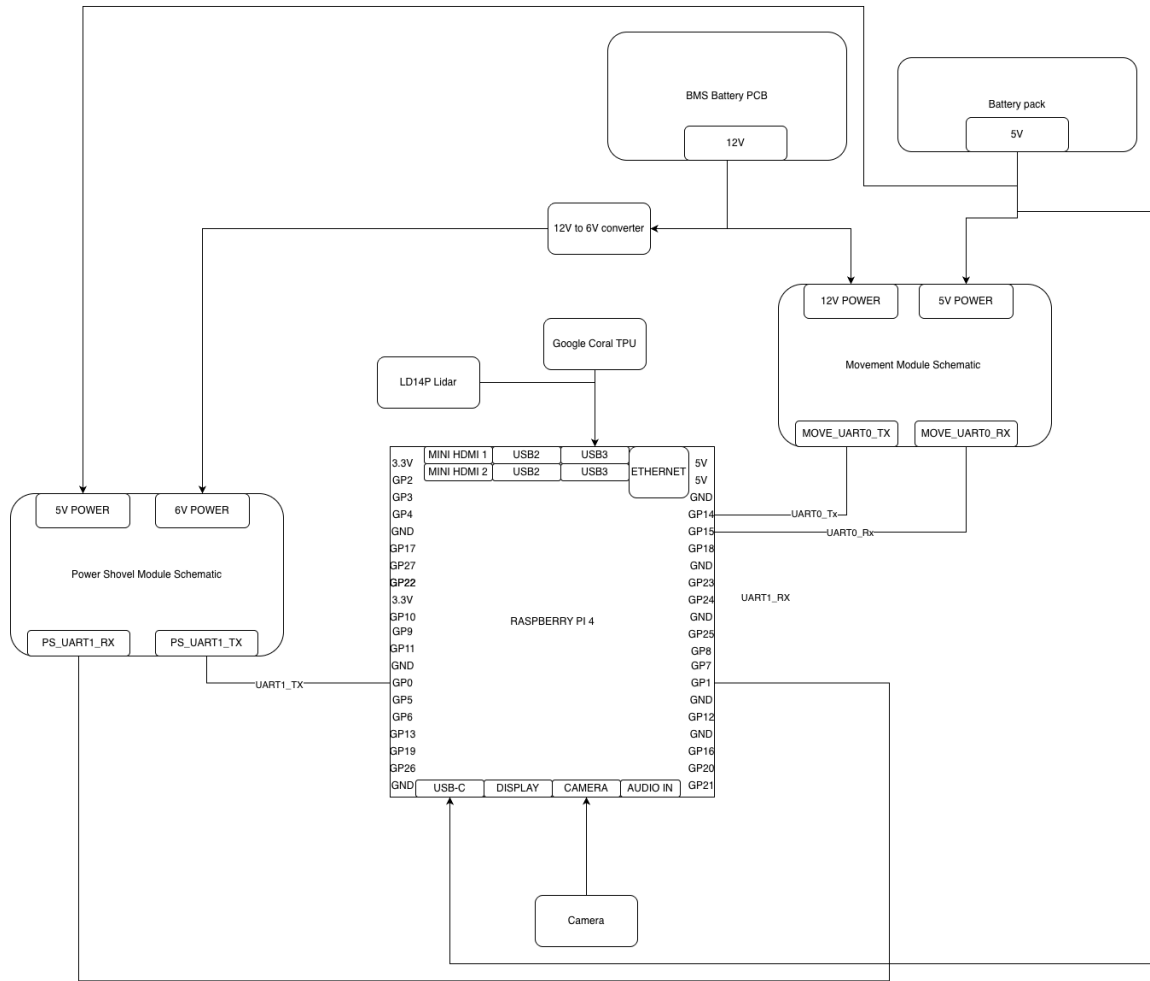


FIGURE 4.3.2.2: MAIN SCHEMATIC

writing TODO

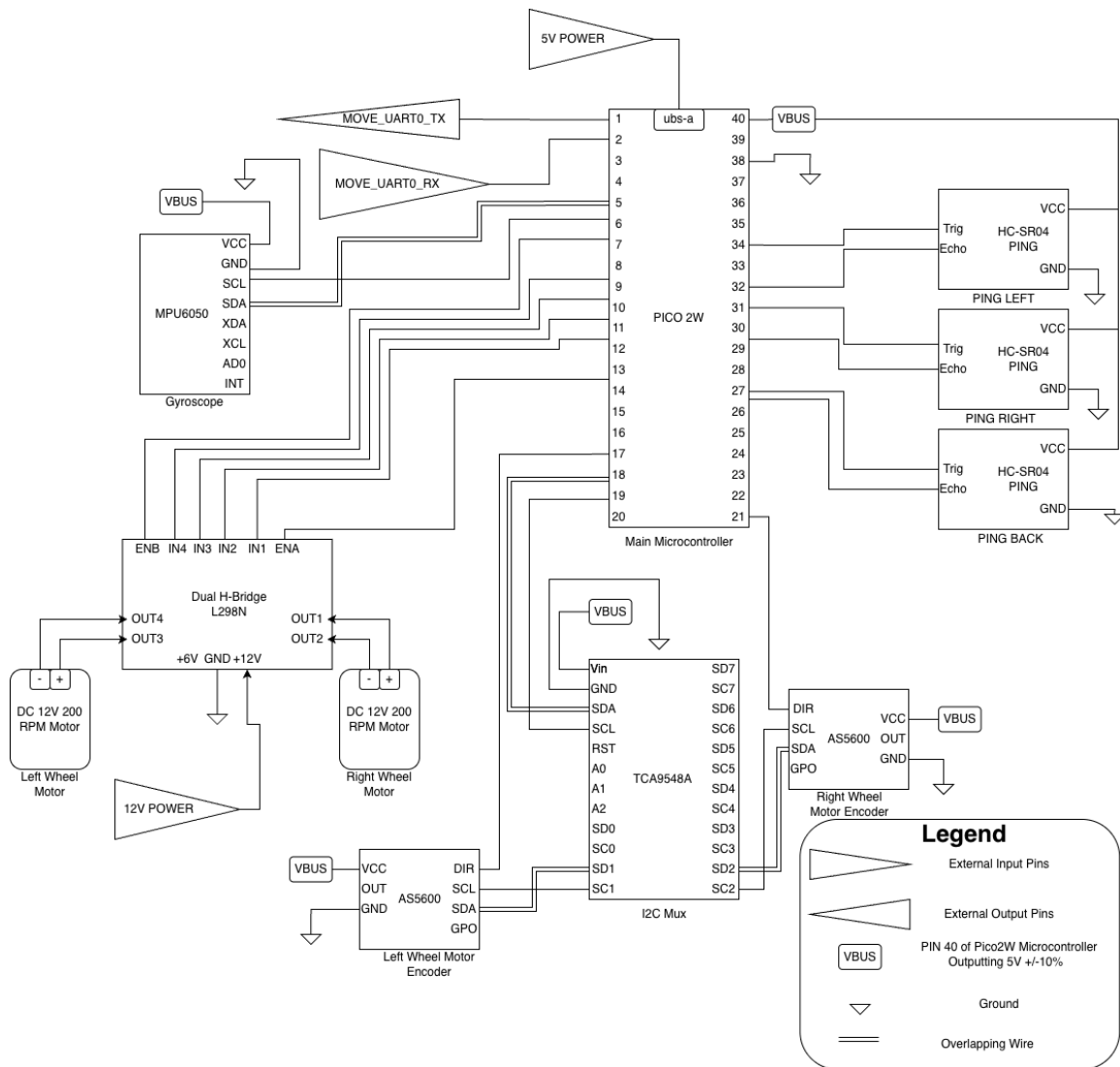


FIGURE 4.3.2.3: MOVEMENT MODULE SCHEMATIC

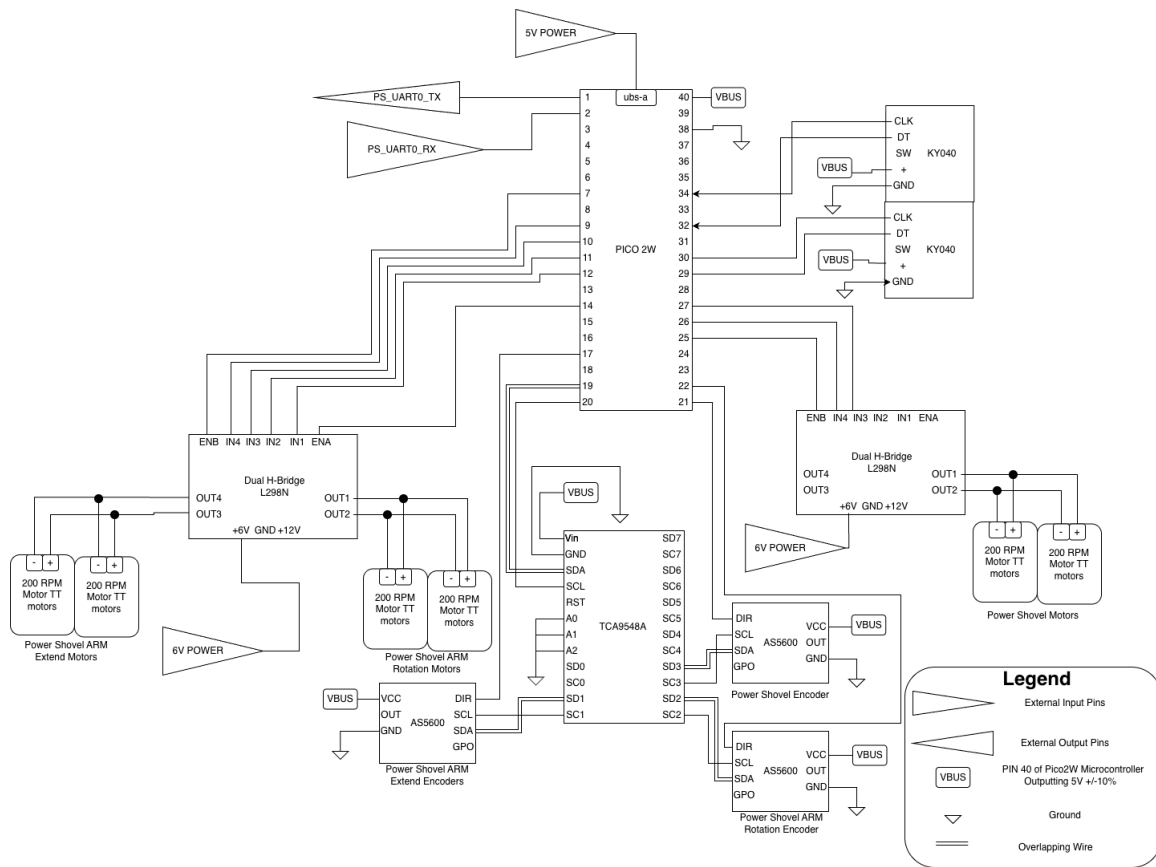


FIGURE 4.3.2.4: POWER SHOVEL MODULE SCHEMATIC

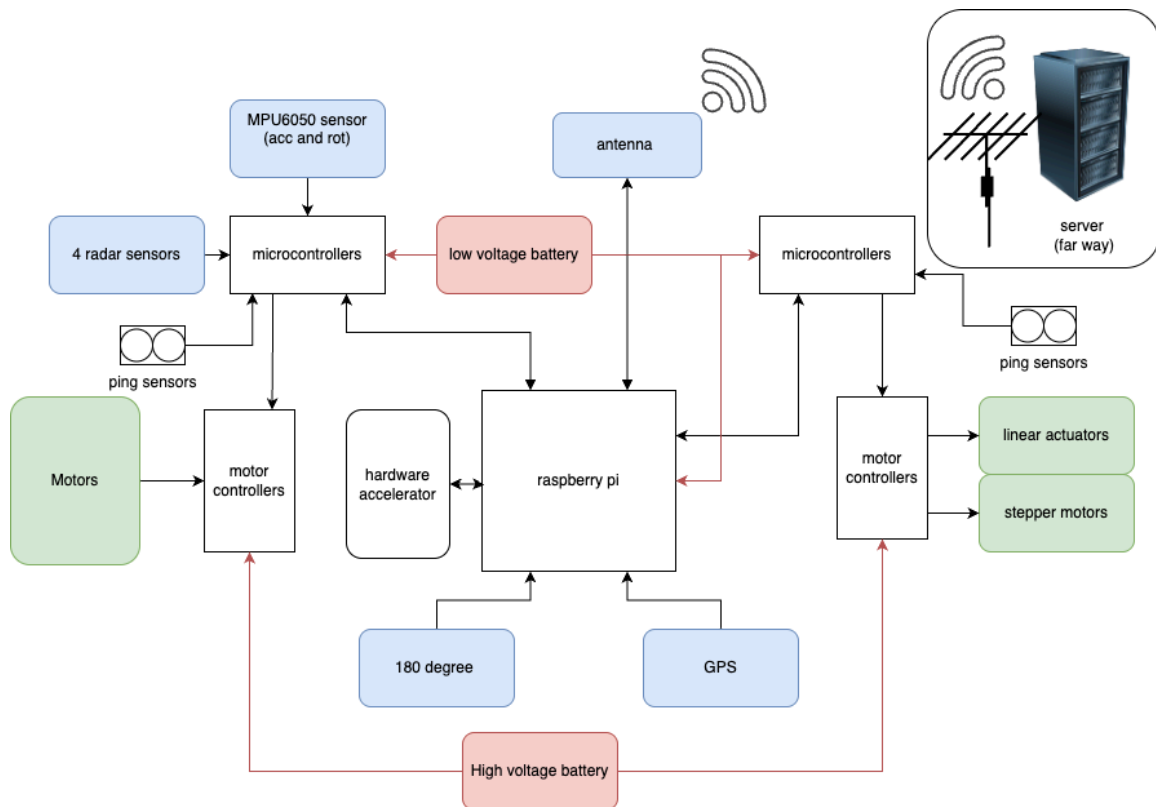


FIGURE 4.3.2.1: EMBEDDED DIAGRAM

We plan to use two motors to move the weight of itself and the robot plus the five pounds of cans and bottles while climbing up a 45-degree incline at least a velocity of 1 m/s. While not being damaged by external movements. So far, we believe two brushless DC motors with a torque of 10 Nm and RPM of 200 or more would be a good pick since it's less likely to stall and can move itself. The motor would be around 24 V and take up 2A, therefore a 24V with over a 7Ah battery would be perfect for the motors. Since two Ah would be needed for that motor and one extra Amp hour for more wiggle room for the Rumble.

The sensors we will be using are 4 radar sensors, a gps system, as well as a 180 degree camera. These sensors will be able to detect people nearby and identify objects with the camera and machine learning and its location. The electronics involved will be a motor controller that will be used for the motors as well as a separate microcontroller to take in all of the sensor data and send it to our main processor, the Raspberry Pi with an hardware accelerated AI module. The raspberry pi will have a few jobs, first it will be used for the camera detection of objects, for this reason we will be using a separate microcontroller for other sensors since the processor will be busy. Secondly it will communicate with our backend server which will tell it where cans or bottles are. Our scooper mechanism will work similar to a dustpan and broom with the linear actuators pushing out and pulling the object into the scooper.

Currently in our system we have multiple of our Rumbles with a charging/docking station and a computer server that connects to all of them. Every Rumble has a docking station where the Rumble can unload the trash collected and charge its battery. The Rumble's main job is to pick up and collect Cans and bottles from the surrounding area

and bring it back to the charging/docking station, without running out of battery mid way. The Server's main job is to gather data of incoming trash being pinged and manage which trash the Rumble should pick up. If the Rumble detects cans or bottles from it's camera it will check if the it has enough room to hold it in it's compactor else it will ask the server for trash location. Next it will see if it's enough battery and space to pick up trash and bring it back.

4.3.3 Functionality

Since the robot is autonomous, our user set up the robot through an interface. The user can set geofencing, where they can put the boundary of the robot where they can operate. Once setup is complete, the robot will roam the plane and look for trash (cans and bottles) via its camera. When it detects trash, it will pick them up with the sweeper. Since this is a robot that takes place outdoors, there can be damage to the robot, and that's when the user has to repair and maintain it. If the robot gets in an accident, for example, by falling or being stuck, it will notify the user that's when the user has to interact with the robot. Lastly, the user should check to ensure the robot is charged. When the robot is complete (reach capacity of the trash compactor) the user should empty the compartment before it is returned.

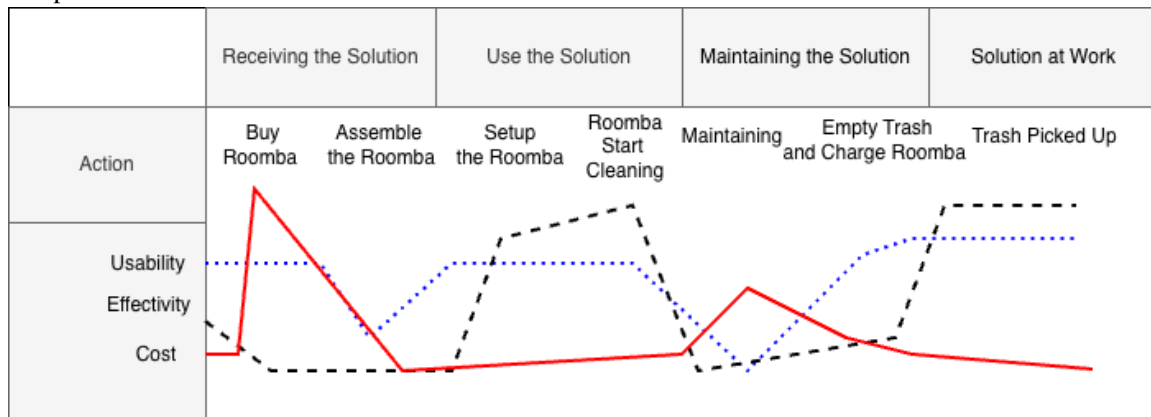


FIGURE 4.3.3: JOURNEY MAP

Above is our Journey Map, a map we are measuring cost in money. Usability is how easy it is to use the robot, and the effectiveness is how much trash is cleaned. Initially when the user buys our product it should be as easy as clicking a button and can cost around 500\$ or more hence the high cost. The next Assembly of the Robot is going to take some time. Then the setup of the Rumble will be done via a mobile app which should make it easy for the user to set up. Next the Robot will start cleaning which will finally start making it effective. There will be time when the robot will need to be prepared and can cost a little and would make it hard to use. Then the user would have to empty it.

4.3.4 Areas of Concern and Development

So far, the current design is low maintenance and easy to repair since they use our cube-like design, which has a detachable compartment for trash, making it easier to clean. Since it has an IPX4 rating, it will be able to have a longer life. Its sensors, camera, and machine learning model can detect objects and trash. It can navigate around the objects with its path-finding algorithm and GPS. It will be connected to the server to find a better path. We have batteries that last an hour, which meets our expectations and High torque motors that can carry five lb of trash. The robot should be able to travel a one-acre radius max with a battery life of three hours and speed of a minimum of one m/s. So far, we have been able to meet most of our requirements.

So far the biggest primary concern for delivering a product would be the durability of our robot in the testing field out in the real world. We will be testing with clients in urban areas close to cars and restaurants, which can get crowded and unpredictable. This can break our robot; if that happens, I am unsure about ETG covering our damage costs.

One of the solutions to address the damages would propose a higher budget from ETG and the Computer Engineering department. This way we can fix and replace components for the robot. Another solution is to ask companies for funding and pitch them the proposal. This way we will be sponsored and fix our robot. But the best solution would be prevention. We would build the robot so that it would be hard to break, and we would test our way as far away from the water as possible and avoid risky situations for the robot.

4.4 TECHNOLOGY CONSIDERATIONS

Some of the distinct technologies we are using are AI, radar sensors, and motors. Some of the strengths of our technologies are that using AI allows for easier object detection and identification while the radar sensors allow us to easily detect people and objects. One of the pros to getting a more powerful motor is that we do not have to worry about the weight of the robot because we will get motors that are more than enough to push the robot and anything within it. Some of the cons are that some of these components are expensive and could be out of our budget, as well as taking more power to run. Some pros of getting a weaker motor is that it is cheaper and will require less power allowing our batteries to last longer. Some disadvantages are that it can carry less load and will force us to get lighter parts for other sections.

5. Testing

5.1 UNIT TESTING

The units that we will be testing are the applications, sensors, microcontrollers, and machine learning models.

Testing sensors can be tested by checking the software written to control and monitor them and seeing if the output makes sense based on what the sensor is seeing. We will use the Python Matplotlib library to plot out the sensor values and compare if they meet our expectations in different configurations under different conditions. If it meets our expectations, we will reconfigure those settings. For example, our radar can be configured to pick up use frequencies and can pick up objects depending on their orientation; one orientation will give more range but will be more narrow, second orientation has less range but is broader. We will graph those two orientations, and depending on which one is better at picking up objects, as shown in the recorded graph, we will pick those.

Each microcontroller and their code can be tested in their stages. First, isolating their files tests the input and output of the functions. Once we have an expected output, we will do a schedulability test, when it runs on an RTOS(real-time operating system), where each task's period is in a "harmonic" series, fits a 69% utilization so that when it runs on Rate Monotonic Scheduling (RMS), the task never misses a deadline. If it doesn't, we will have to optimize or halt some of the functions. Next we will test the whole microcontroller serial input and output ports with this code. This will allow us to have a real-time system for the sensors and motors.

With our machine learning model, we will run through testing datasets to test the accuracy of the model. Testing sets are a subset of the training data that the model hasn't been trained on or has ever seen. Usually, in a Testing set, 20% of total training data is used in the same dataset to eliminate bias and get an accurate accuracy rate. Testing can be done in Python by various libraries; a popular one is Pytest, but Tensorflow (a popular Python library for machine learning) has an in-built function for testing, which should be enough. But to make our model ready for the real world, we will set it on various datasets on Kaggle (a platform for machine learning datasets and models) that specialize in trash classification, where there are around 21 datasets made for computer vision trash classification with way over thousands of images.

For training, our machine learning is in charge of directing the robot to the trash detected by the trash-detecting module by doing field tests. It will require placing a trash trial object on a field and precisely measuring the distance and angle to the robot, seeing how accurate the module is, and then, based on the results, training the model. Another way to achieve this is by using a Python script that simulates the output of a real camera and AI model. It creates blank frames (pictures) using OpenCV and manually adds simulated bounding boxes to represent things it has recognized, such as rubbish. To test how your direction logic (e.g., move left, right, or forward) reacts depending on the object's placement in the frame, we will provide the size and position of these boxes. We will test if your robot chooses to turn or move appropriately by using these bounding boxes and labels, which appear on the screen just like actual AI output.

We are working with an iOS application that connects to a PostgreSQL-powered Spring Boot backend over HTTP APIs. We use JUnit to test the backend and Mockito to unit-test specific components, such as controllers and services, to guarantee quality. We use TestContainers to spin up a genuine PostgreSQL instance for integration testing, which enables us to confirm database interactions in a separate setting. API endpoints are verified with RestAssured or MockMvc. On the

iOS side, we use URLProtocol to simulate API replies and XCTest to unit-test networking and business functionality. We execute the backend, usually using Docker Compose, and use XCUITest to mimic user flows and confirm real-time communication between the application and the server for complete end-to-end validation.

5.2 INTERFACE TESTING

For our interface testing, we have three major pairs of parts that connect. These will be the user interface and server, the server and Raspberry Pi, and the Raspberry Pi to the microcontrollers. The Raspberry Pi and microcontrollers will communicate to send motor and sensor data to the Raspberry Pi. We can test this by sending a command to the Raspberry Pi and see if the motors respond, also we will test by reading out the sensor data to ensure it is correct.

The second interface is between the mobile and web apps and servers. To ensure these components communicate properly, we will use two unit testing classes for each side of the connection. The one for IOS will be using URLProtocol for simulation to test our networking. On the server side we will be using spring boot with a MySQL database, this can be tested through normal Java unit tests that will include connecting to the robot.

5.3 INTEGRATION TESTING

Some of the parts that need integration tested are our microcontrollers for sensors and motors sending data to the Raspberry Pi. We could test this by having the microcontroller have a test function that will generate test data to send back to the pi. Once this has been done, we can use the software on the Pi to verify that the data sent is the same as the test data stored on the Raspberry Pi.

To have the server support for multiple robots. We can simulate a connection to the server, and the real robot avoids acting up on it. We can use Webots, an application for simulated robots that has a feature to post HTTP commands to the web. We can use Webots to mock multiple robots in the world by making a virtual twin of our robot. This way, the virtual robots can talk to the server, and the main real robot can act upon it. For example, if the virtual robot detects trash and pings it to a location, the real robot could go there and try to find it. If a virtual robot is in the way of the real robot in terms of global position, the real robot will move out of the way, etc. This way, we can have the server and app support multiple robots while not actually building real robots, keeping it cheap.

Integration tests of machine learning models will focus on the performance and accuracy of the model. We need to test if the camera we pick is good enough for the model to detect trash. For this, we will have a small collection of trash caught by the camera in some of the test runs (less than 100 images). Then, we will run the models through those test sets and compare the accuracy rate to the normal accuracy of the models. We want it to be around $\pm 10\%$ difference from the model's usual accuracy tests on which the dataset test set is being trained.

5.4 SYSTEM TESTING

For system-level testing, we will put the robot through a set of tests. For the unit tests, we will test the robot's trash detection and pickup. For interface testing, we will test if the data is getting from the sensors to the Raspberry Pico so that we know that the robot can detect obstacles. Another integration test will be using the Raspberry Pi's camera to identify trash with the Machine Learning model. As for the integration tests, we will need to ensure that the robot is connected to our server and that data can be sent and received by both. A tool we can use to test would be Postman for the server/robot connection. All of these functions are requirements of our project, we need it to pick up trash and be able to navigate through the use of the sensors. Will also make sure the robot is capable of detecting fast-moving objects like cars and avoiding them accordingly. Lastly,

we want the robot to decide intelligibly when to head back to the docking station depending on the battery charge, weather, and distance needed to travel.

5.5 REGRESSION TESTING

Simulation by Webots: For movement it is better to test if the robot will move in a simulation rather than praying it will work in the physical sense. This will save us time when it comes to debugging the robot, adding design changes, and verifying everything before throwing it into the final model. It is best to test each part individually for the physical parts. For example, for the motors, our data will be more accurate if you test each motor's RPM to see if there are any malfunctions. Saves a lot of time and headaches. You should always trust but verify the products. Lastly, simulations will benefit when we have to take user feedback and apply changes to the robot.

Requirements do drive our regression testing as we will have to make sure that we still meet our requirements that have been previously implemented while adding new parts to achieve different requirements.

5.6 ACCEPTANCE TESTING

We can demonstrate that our design requirements are being met through various testing of battery life, durability, and capacity of trash. An easy way to do this would be to run the robot with enough trash in its area to fill it up or keep it running for over three hours. Then, we would check how long the robot has run, how much trash it actually picked up, and if it is damaged in any way. Our client would be the one to decide if the requirements are acceptable and will see the results of the test to ensure that we have met all of them.

5.7 User Testing

For user testing, there is a survey, prototype analysis, and user reports. The survey will ask potential users if they currently use any similar product, and what technical aspects they would like to see in this project. This early survey will ensure that the user gets a product that they will be able to use and happy with the performance of. The prototype analysis will determine what aspects of the current working design need to be addressed and improved upon, focusing on usability and performance. Finally, the user reports will determine what needs to be improved for a potential second version of the robot. This will be data received from running the robot such as how long the robot is in use, what types of trash it picked up, how frequently it fills up, and how many stops the robot makes. These can then be used as new requirements to improve upon the current design.

5.8 RESULTS

Different motors and how they would behave with different weights and inclines. So far, we have found that an RPM of 100 or higher, a torque between 100 oz-in to 300 oz-in, and a payload of

rolb can get a speed above 1 m/s on a 45-degree incline.

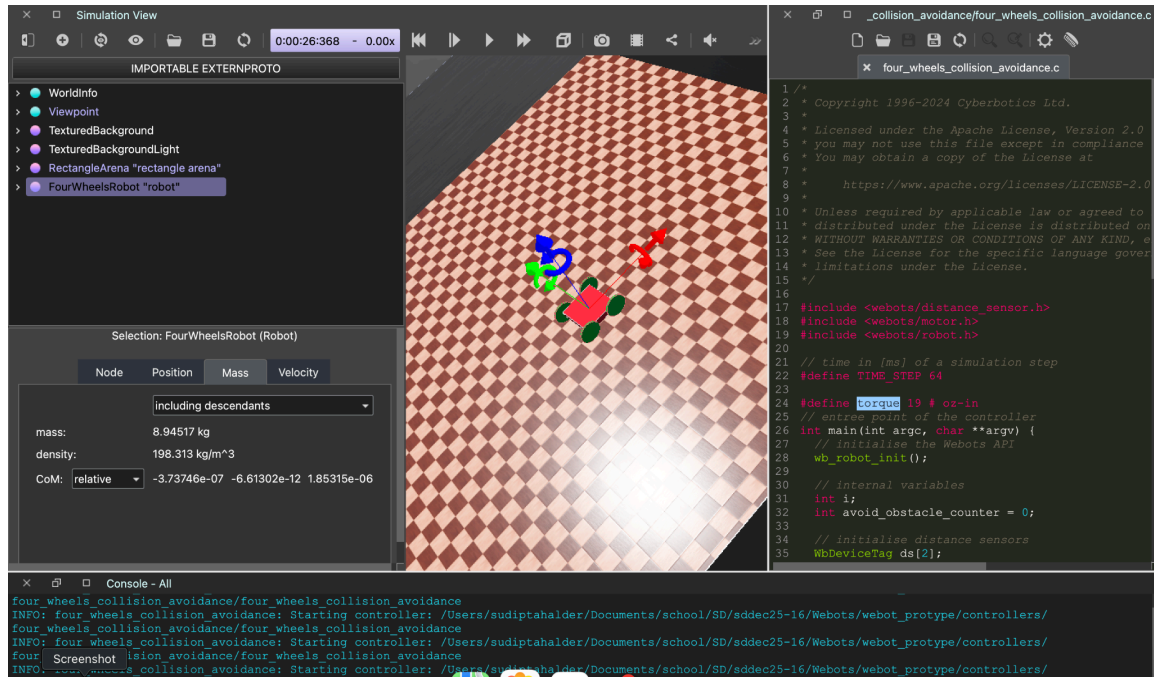
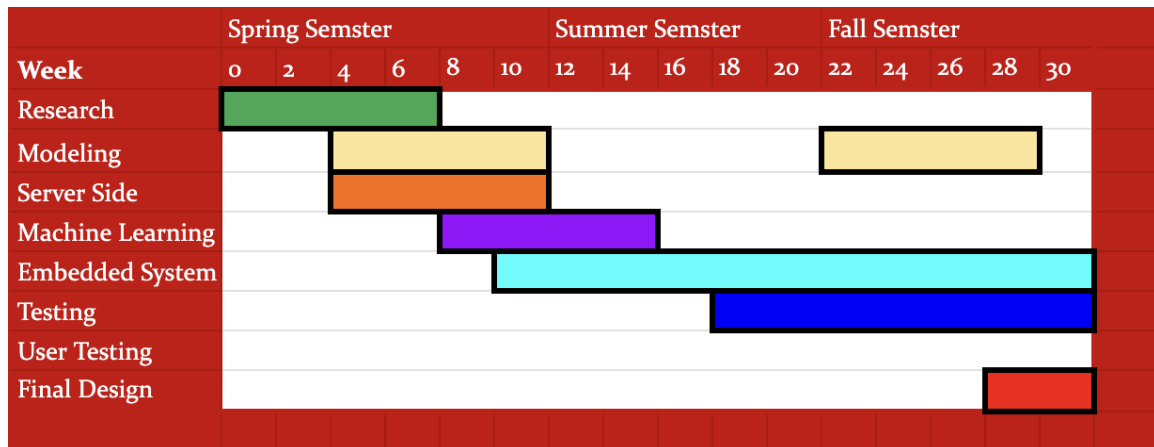
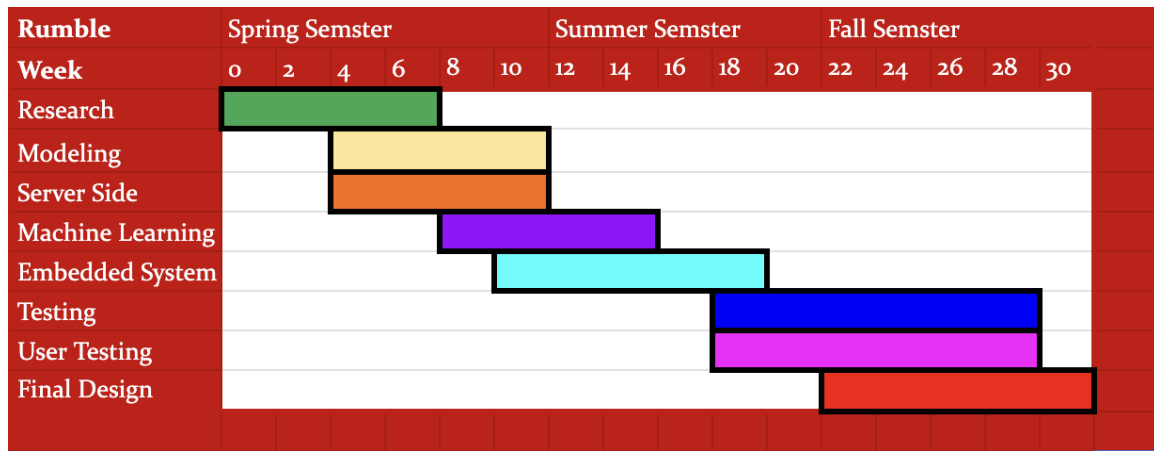


FIGURE 5.7.1: USE OF WEBOT

6. Implementation



6.EMBEDDED SYSTEM

6.MACHINE LEARNING

6. CAD

So far the team has created CAD models, an application, a backend server, and used Webots to simulate the robot.

For the CAD models we have had multiple iterations of them. The most recent one is shown in *Figure 6.1*. This is a way to implement space requirements without building the robot as well as determining how objects fit into the pickup mechanism.

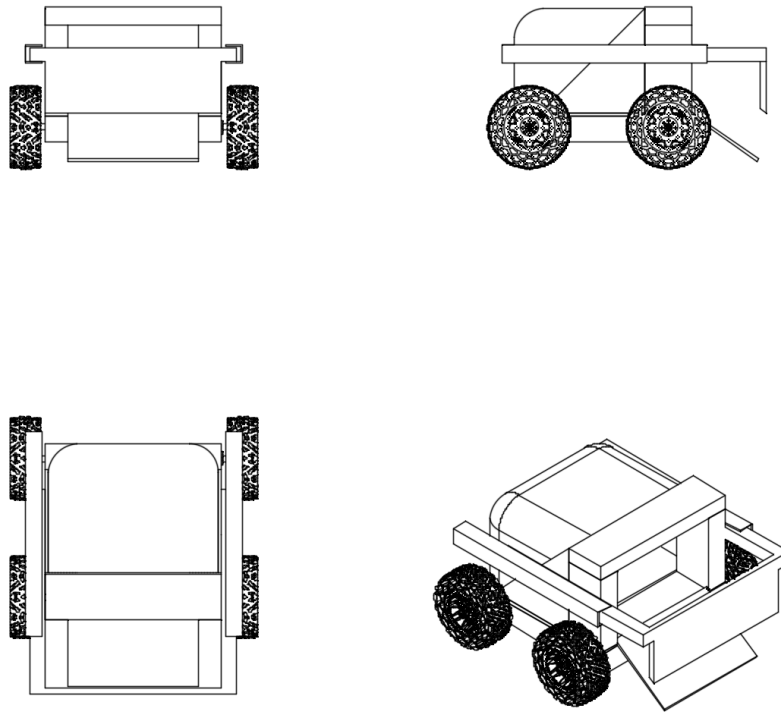


FIGURE 6.1: CAD MODEL

In Webots we have started creating a virtual twin to the robot we will be using in the field in testing. Since we have recently completed our robot model in CAD we haven't imported it to Webots yet. But we have simulated some of the potential motors we will be using. Which you can see in *Figure 5.7.1*

A Basic Server has been created with the functionality of having multiple accounts for users and robots and other controllers for interfacing with different user interfaces.

6.SERVER

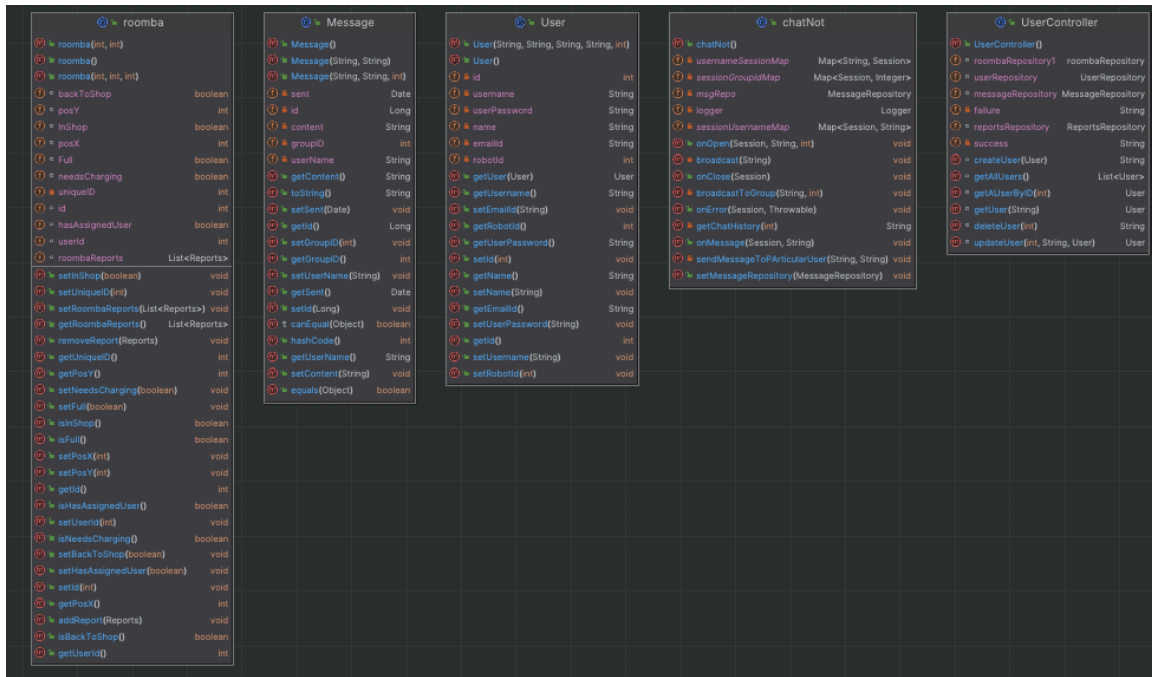


FIGURE 6.2: BLOCK DIAGRAM ACCOUNTS ON SERVER

6.FRONTEND

The RUMBLE frontend was implemented as a modern application using React 18, leveraging functional components and hooks for state management throughout the application. The architecture follows a component-based design pattern with clear separation of concerns: authentication components handle user login, registration, and password recovery; dashboard components manage robot monitoring and fleet statistics; settings components enable profile management; and a maps component integrates Google Maps API for real-time robot location tracking. The application uses React Router for client-side navigation with protected routes that restrict access to authenticated users only, implemented through a custom PrivateRoute wrapper component. Global authentication state is managed via React Context API, providing a centralized AuthContext that handles user sessions, JWT token management, and localStorage persistence across the entire application. The styling system combines Tailwind CSS utility classes for rapid UI development with custom CSS animations defined in a global stylesheet, creating a cohesive dark-themed interface with glassmorphism effects, gradient overlays, and smooth transitions that enhance the user experience.

Communication with the Spring Boot backend is handled through a centralized API service layer that manages all HTTP requests using the Fetch API, including comprehensive error handling for network failures, authentication errors, and server-side validation issues. The service layer implements RESTful API consumption patterns with automatic token injection for authenticated requests, response parsing for both JSON and text formats, and graceful error recovery that provides meaningful feedback to users. The dashboard displays real-time robot statistics including active/maintenance counts, battery levels, and trash collection metrics, all fetched from backend endpoints and dynamically updated when users interact with robot controls. The Google Maps integration uses the @react-google-maps/api library to render interactive maps showing robot positions with color-coded markers indicating operational status (active, charging, maintenance), complete with info windows displaying detailed robot information. Form validation is implemented

client-side using custom validation functions that enforce password strength requirements, email format verification, and input sanitization before submission, with real-time error messaging that guides users through the authentication and profile management workflows while maintaining security best practices throughout the application.

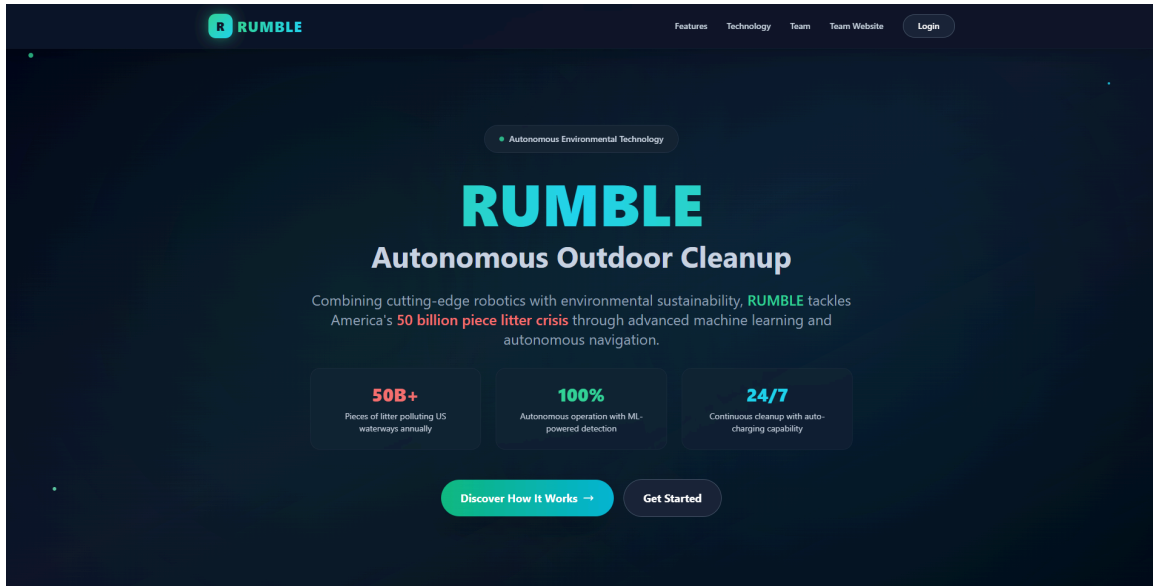


FIGURE 6.3.1: FRONT-END LANDING PAGE

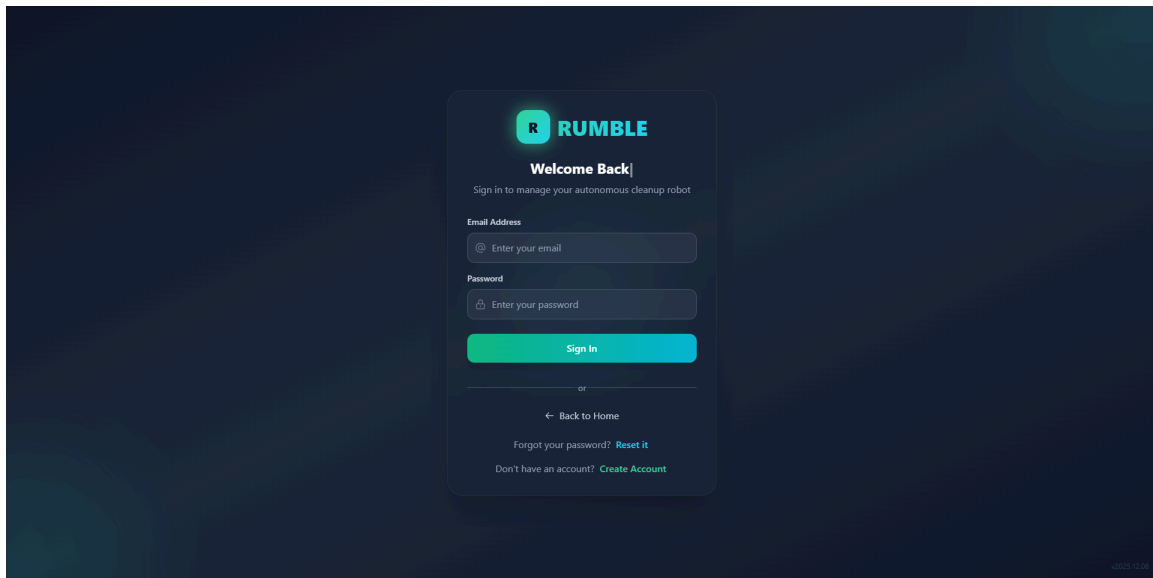


FIGURE 6.3.2: FRONT-END LOGIN PAGE

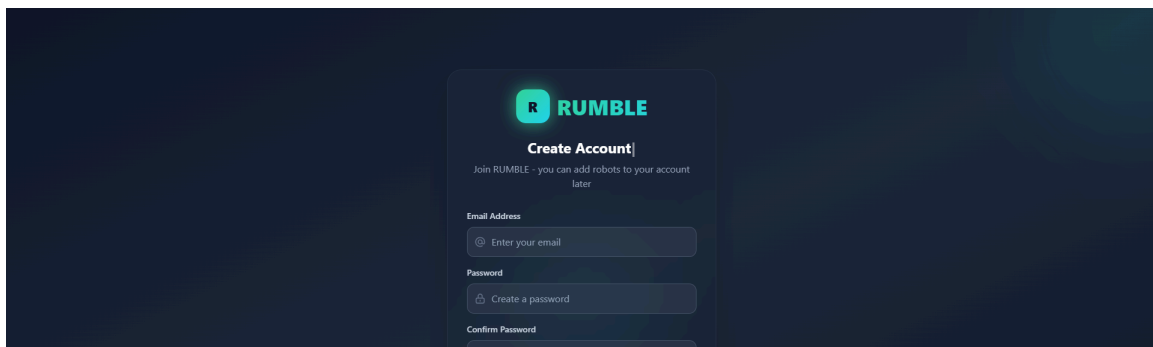


FIGURE 6.3.3: FRONT-END CREATE ACCOUNT PAGE

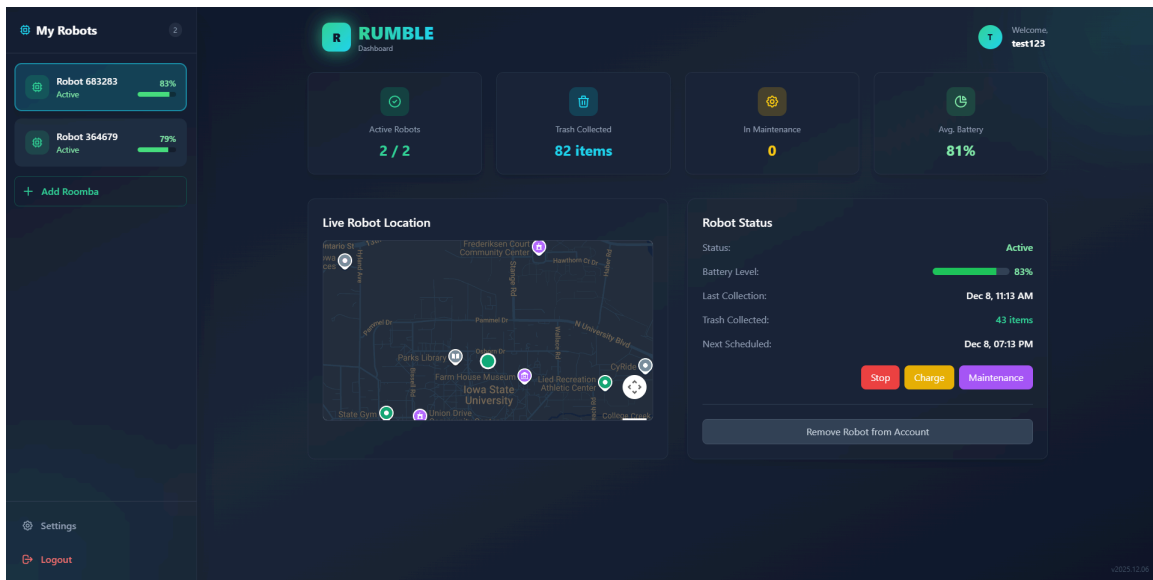


FIGURE 6.3.4: FRONT-END DASHBOARD PAGE

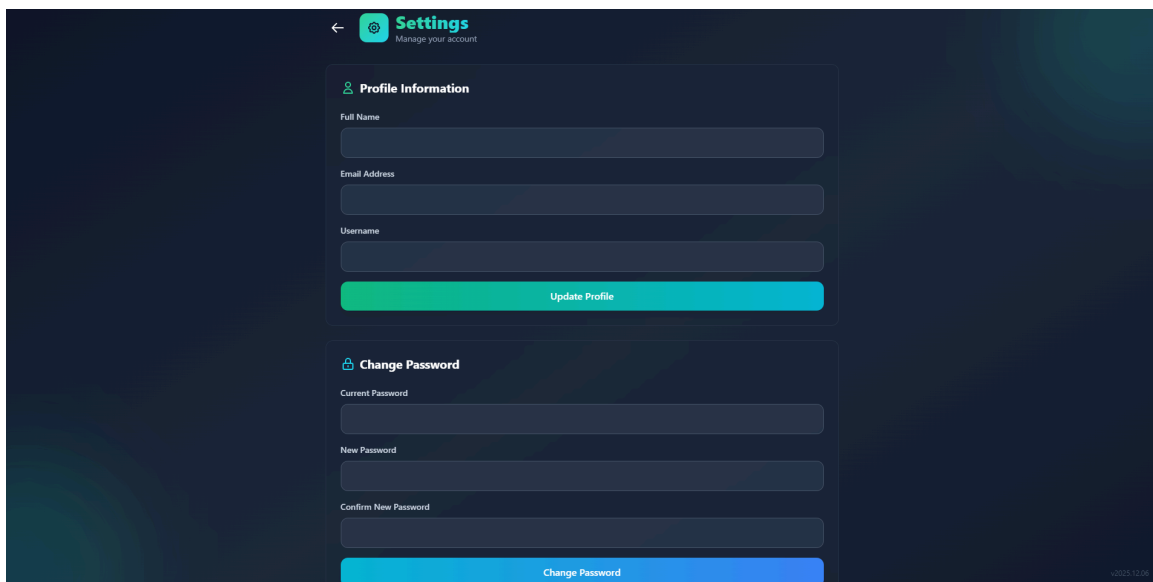


FIGURE 6.3.5: FRONT-END SETTINGS PAGE

6. DESIGN ANALYSIS

We have tested the object detection and have used Webots to start simulating our design. We have CAD modeled our design and it should fit our requirements. We have not been able to test parts such as our sensors, motors, and motor controllers. So far the parts we have tested show that it works. The object detection part works from our raspberry pi with a camera. For our future design plans we need to test our other components and see if they work for our project.

TODO

7. Ethics and Professional Responsibility

7.1 AREAS OF PROFESSIONAL RESPONSIBILITY/CODES OF ETHICS

Area of Responsibility	Definition	IEEE Code of Ethics
Work Competence	Completing tasks efficiently and with a standard of quality.	“7. To seek, accept, and offer honest criticism of technical work, to acknowledge and correct errors, and to properly credit the contributions of others.” This relates to us providing proper criticism for our work throughout the project.
Financial Responsibility	Solving the problem while keeping prices reasonable and within budget.	4. To reject bribery in all forms.” Staying honest to our budget constraints as a group and remaining responsible.
Communication Honestly	The ability to communicate with the team and users while maintaining integrity.	3. To be honest or realistic in stating claims or estimates based on available data.” This relates to our group providing valid sample data and not misrepresenting the values.
Health, Safety, Well-Being	Maintaining good mental health within our team and not overworking one group member. Splitting up the work accordingly.	“8. To treat all persons fairly regardless of such factors as race, religion, gender, disability, age, or national origin.” This ethical statement aligns with the well-being of our team and its members.
Property Ownership	Respecting the material and parts purchased especially when given a budget out of someone else’s pocket.	“8. To treat all persons fairly and to not engage in acts of discrimination based on race, religion, gender, disability, age, national origin, sexual orientation, gender identity, or gender expression.” This ethic also applies here to allow for little judgment with property and ideas.

Sustainability	Respecting the usage of materials and reducing wear and tear for our project.	“5. To improve the understanding of technology, its appropriate application, and potential consequences.” Respecting material usage reflects understanding how to apply technology responsibly while considering its long-term consequences.
Social Responsibility	Provide a safe, easy to use product that will be environmentally friendly and provide guidelines.	“10. to support colleagues and coworkers in their professional development and to support them in following this code of ethics.” Being communicative and speaking with members of the team ensures social responsibility.

Our group was efficient in “Health, Safety, and Well-Being”, as we made sure to make sure everyone in our group was doing okay mentally. We understood that some weeks are going to be busier than others, and if one of the members were following behind in their work, there was always another member ready to help to get the project or assignment done the correct way. The part we could improve on is the communication honestly which sounds counter intuitive, but as a team we could have worked more on communicating when we were confused or needed help. We as a team had a lot of great ideas, but at times it was hard to communicate which idea we actually wanted to go do.

7.2 FOUR PRINCIPLES

	Beneficence (Promoting Good)	Nonmaleficence (Avoiding Harm)	Respect for Autonomy	Justice
Public Health	Clears public surface areas which can reduce hazards when driving or walking.	This reduces waste and helps eliminate some of the public trash problem that is slowly infecting our earth.	We have an app that will allow the users to communicate with the robot if there is a problem at hand.	This allows public workers to focus on their great plans instead of something that shouldn't be an issue in the first place.

Global/Culture	This robot will help clean up streets around the world which is good because even though we live in our respected countries we still all share one earth.	Our robot will be able to adapt to the trash around them which means here we use a lot of aluminum bottoms but in other countries they might use glass and our robot is smart enough to pick up both.	We allow the users to decide the area that the robot will clean. Which allows them to determine how they want to integrate our product.	Cleaning the earth as a whole will benefit everyone. Not just one country but the world as a whole.
Environmental	Improves environmental health by cleaning up trash outdoors.	We are fully electric which allows us to not add significantly to the carbon emissions.	Our robot should detect what to pick up and what not to pick up. It should also be able to determine the path that it will take, meaning it should avoid plants and trees and other living things.	Ensure environmental benefits around the world. We are doing this project not because we are lazy and don't want to pick up after ourselves we are doing this to ensure the world gets cleaner.
Economic	One of our goals was to make our robot cheap so it is more affordable for the everyday users meaning it shouldn't break the bank.	Electricity is universal around the world, so we are going to use a charging port that is widely available around the world. If we had more time we can discuss about adding additions to our project like solar panels for the locations that don't have the benefits of electricity.	Our compacted robot allows our customers to not have to worry about significant price changes. Because our robot is simple and cheap.	Ensures economic standardization allowing customers around the world to not have to worry about a massive change in operational costs for the robot.

FIGURE 7.2: FOUR PRINCIPLES

The environmental and economic principles are the most important to us as a team. It is our goal to make a cheap, but effective way to clean the earth. We as a group don't want to take advantage of trying the better the world by making an expensive robot. If we wanted to can we yes, but what is the purpose of making a robot that can better the world if only a small percentage of the world can help be the solution to the problem. The best way we can accomplish this goal is by staying within our budget. We know first hand that components right now are expensive, so if we can find the best products for the lowest cost that our customers can afford, then we can really make a difference.

7.3 VIRTUES

As a team we determined three virtues that are most important to us. These virtues are patience, ambition, empathy, honesty. We have listed these virtues below along with our own definitions and how we support them.

- Patience: The ability to tolerate difficult assignments that are assigned to you, and also capable to collaborate with other team members.
 - We support this virtue by always responding fast to fellow teammates when there is a problem. We also allow a lot of time for design and encourage taking breaks when you feel like the work might be too much for you at that time.
- Honesty: The ability to speak to your team when there is a disagreement, problem, or even suggestion that you think will better the team or the project.
 - Our team doesn't believe that there are bad ideas. Everyone has the chance to speak their mind and explain why they think that way and we are respectful enough that if the idea is good we will implement it, but if it is a better solution we talk it through until we make the decision we believe is the better of the two.
- Ambition: The desire to achieve success in our project or any testing we want to achieve.
 - We completely support this by attempting regular meetings and to be there on time. We also set up a voice channel to discuss ideas at a later time if something comes up that doesn't allow a team member to make it to our meetings.
- Empathy: Having passion in your work. Doing this not only because it is assigned to us, but because you truly believe in the work we are doing.
 - As a team I believe we are all very empathetic towards this project which allows us to bounce great ideas off constantly to better the world. This virtue allows us to see different perspectives of the same issue at hand.

Below, we have all reflected our own personal strengths and weaknesses when it comes to our virtues.

Mitchell Owen:

The virtue that best describes me is Ambition. Most of my work involves hardware, so this semester I have a crucial role. Last semester we focused a lot of digital testing, but this semester we have to focus more on making that digital model into a physical one. That means that there isn't a lot of room for error on my part. I am huge on teamwork and making sure everyone is on the same page. With the short amount of time we have left together we have to be able to communicate when we have issues with our project to come with a final product in time for us to present. I hate letting my team down because a failure in one department for this product is a personal failure to me. This is a team project and I want everyone to be successful and to stay busy, and I believe that we accomplished that this semester.

One area of improvement that I believe I need to work on is patience. I hate when I can't solve a problem. Luckily I am good at communicating and being honest, so when I have a problem or I am really struggling mentally I let my team know and they are so supportive. One way I can improve my patience is communicating with my teammates sooner rather than later which will give me more time to solve the problem instead of procrastinating the issue at hand until it is almost too late. This virtue is important to me because in my degree I won't always just have the solution. This requires me to take a step back and start from the beginning to better understand the problem then approach it a different way.

Sudipta Halder:

The virtue that best describes me and is essential to me is Honesty. Honesty allows us to build trust among team members, advisors, and clients. It also allows us to communicate about the needs openly, wants and problems about the project that can be addressed. So, in this project, Honesty is a virtue I have been demonstrating continuously. Whether it comes out to a problem I have created in git, acknowledging my team and advisor know the high cost of components, and Mistakes. I have always come forward and admitted my mistakes, brought up problems, and more. This has allowed us to go to where we are today. We have a solid foundation of our design and we will continue to excel in the fall semester and complete our robot.

Patience is one of the virtues I have least demonstrated out of the four virtues. This is because most of the assignments in 491 aren't too bad. However, I struggle with writing assignments and take less interest in them. Moving forward, I aim to complete some of the more complex tasks, pull through and complete them as early as possible, and put quality work towards them.

Brodie Bates:

The virtue that best describes me is empathy. This virtue is important to me because it helps me better understand my teammates and determine how to work with them better. This semester I have demonstrated it by working well with my teammates and helping them out where I can.

One of these virtues that I have room for improvement on is ambition. Ambition is important because it is what drives the project. Having ambition leads to better work ethic and an overall better result. Some things that I could do to improve on my ambition is to make it to meetings on time as well as messaging and talking more in the discord.

Om Shukla:

Honesty is the virtue I have demonstrated most in my senior design work. I've been transparent about my progress, acknowledged mistakes when they happened, and communicated openly with my team about challenges. This virtue is important to me because it builds trust and accountability, helping to create a team environment where everyone can rely on each other and collaborate effectively.

Ambition is a virtue I value but haven't fully demonstrated during this project. While I've met expectations, I haven't consistently taken initiative to go beyond the minimum or push creative boundaries. Ambition matters because it drives innovation and progress. To improve, I plan to take on more leadership in technical areas and bring forward bold ideas that can enhance our final product.

Daniel Mauricio Pinilla :

Ambition is a virtue I have fully demonstrated during this project. I have never said no to more work, instead I have tested myself to new heights. I have created an IOS mobile app when I wanted just basic HTML. I have made 3d cad prototypes before we even set our requirements. I completed a server before we even started working on the robot.

Honesty as a virtue I value but haven't fully demonstrated during this project. I am transparent with my work and build trust with my team members. But the only reason I haven't fully shown my Honesty in this project is because my communication is lacking at time. Since I am busy with my job outside of school where I often have to be away from discord. Many time I miss a lot of messages from my team. My goal is to improve that by sharing my phone number instead that way I know what my team needs from me.

8. Closing Material

8.1 Conclusion

So far, we have created a model for how the robot should look and function conceptually, as well as start on an application to connect to the robot and issue commands. Our current goal is to get parts and start assembling the robot. So far most of the parts have been ordered, however some parts were either too expensive for the budget or unavailable. These setbacks made the team redesign and plan out what to order. In a future design iteration, looking into price and parts early would allow for an easier implementation process. Although running into these issues early will allow for a smoother implementation process.

9. References

[1] Pixar Animation Studios, "WALL-E," Pixar Animation Studios. [Online]. Available: <https://www.pixar.com/wall-e>. [Accessed: May 4, 2025].

Pixar Animation Studios

[2] Novelte, "Novel te Outdoor Cleaning Robot S330," YouTube, published Oct. 2024. [Online]. Available: <https://www.youtube.com/watch?v=Pe2W4m5e8CA>. [Accessed: May 4, 2025].

YouTube

[3] iRobot Corporation, "iRobot®: Robot Vacuums and Mops," iRobot Corporation. [Online]. Available: <https://www.irobot.com/>. [Accessed: May 4, 2025].

irobot.com

[4] IEEE Standard for Autonomous Robotics (AuR) Ontology, IEEE Std 1872.2-2021, IEEE, Published May 12, 2022. [Online]. Available: <https://standards.ieee.org/ieee/1872.2/7094/>. [Accessed: May 4, 2025].

IEEE Standards Association

[5] Standard for Human Intentions and Artificial Intelligence Alignment in Autonomous Driving Agent, IEEE P3474, Active PAR (approved May 22, 2024). [Online]. Available: <https://standards.ieee.org/ieee/3474/11639/>. [Accessed: May 4, 2025].

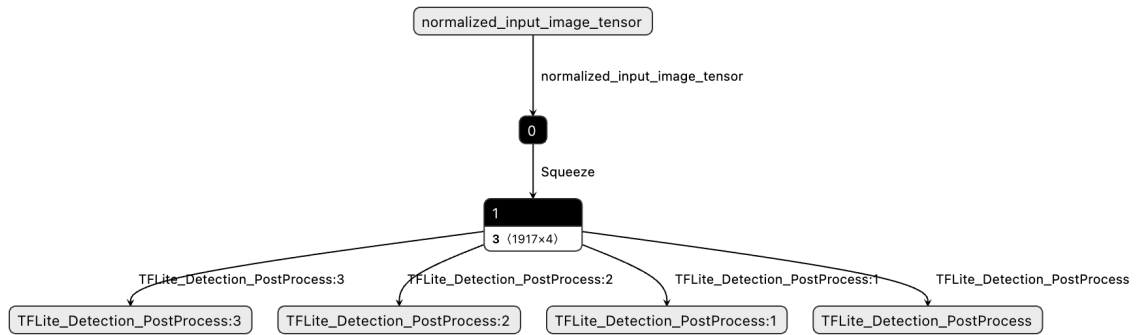
IEEE Standards Association

[6] IEEE Recommended Practice for Electronic Power Subsystems: Parameter Definitions, Test Conditions, and Test Methods, IEEE Std 1515-2000, Published Sep. 15, 2000; Inactivated Nov. 7, 2019. [Online]. Available: <https://standards.ieee.org/ieee/1515/2255/>. [Accessed: May 4, 2025].

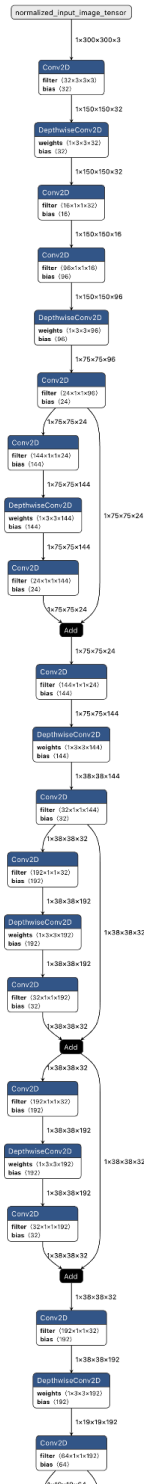
[7] Keep America Beautiful, *Litter Study Summary Report*, May 2021. [Online]. Available: https://kab.org/wp-content/uploads/2021/05/Litter-Study-Summary-Report-May-2021_final_05172021.pdf [Accessed: May 4, 2025].

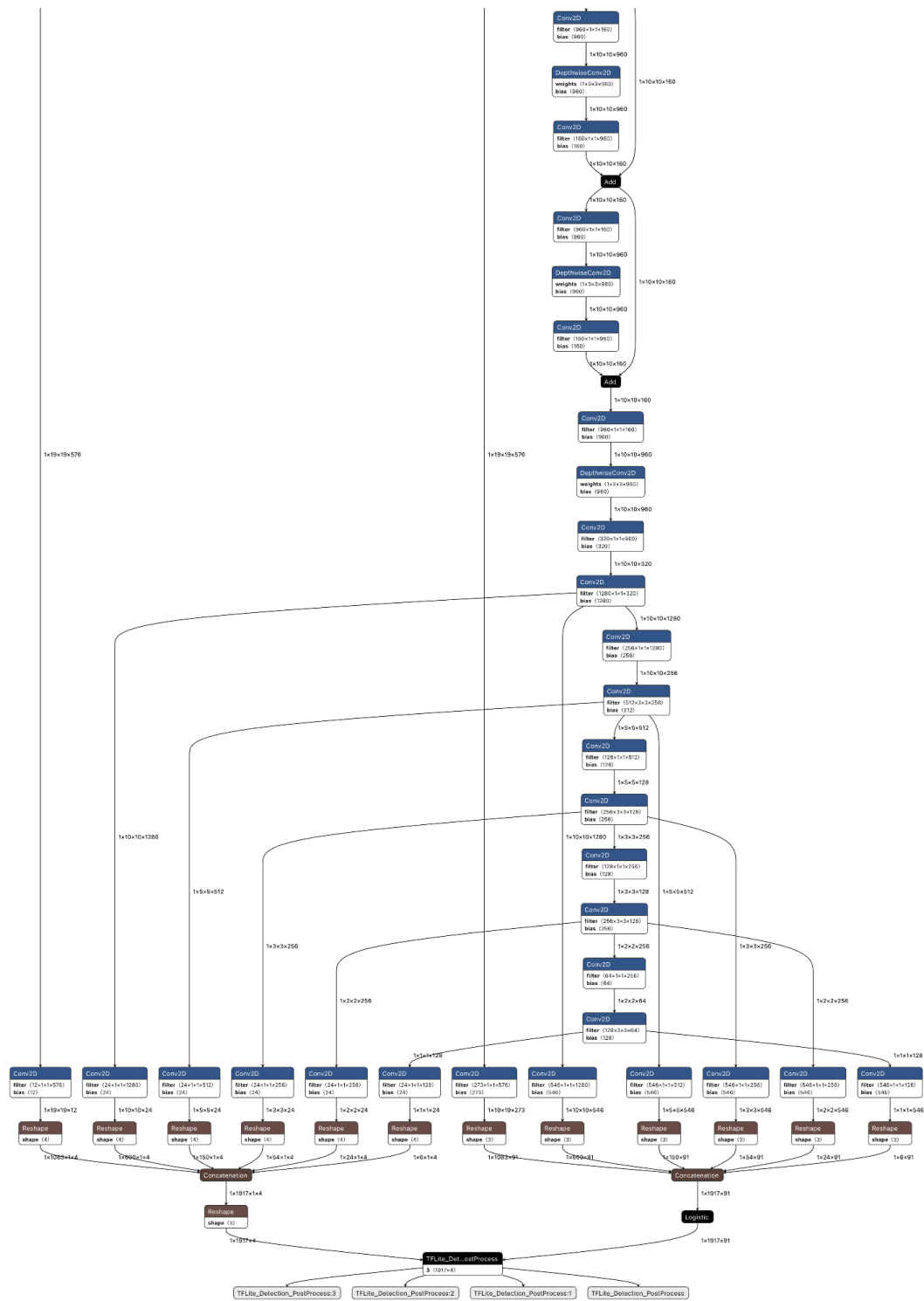
10. Appendices

10.1 APPENDIX: SSD_MOBILENET_V2_COCO_QUANT_POSTPROCESS_EDGETPU.TFLITE



10.2 APPENDIX: SSD_MOBILENET_V2_COCO_QUANT_POSTPROCESS.TFLITE





11 Team

Complete each section as completely and concisely as possible. We strongly recommend using tables or bulleted lists when applicable.

11.1 TEAM MEMBERS

MITCHELL OWEN, BRODIE BATES, OM SHUKLA, DANIEL VERGARA PINILLA, SUDIPTA HALDER

11.2 REQUIRED SKILL SETS FOR YOUR PROJECT

Development on Raspberry Pi

Embedded Systems Design

Cad modeling

PCB design

Website development

App development

Servers

11.3 SKILL SETS COVERED BY THE TEAM

Raspberry Pi Application- Daniel

Embedded Systems Design: Sudipta, Brodie, Mitchell, Daniel

Cad modeling - Mitchell, Sudipta, Brodie

3D ROS(robot operating system) simulation- Sudipta

PCB design - Mitchell, Sudipta, Brodie

Website development: Om

App development: Daniel

Servers: Om, Daniel

Machine Learning Development and Implementation: Sudipta, Daniel

11.4 PROJECT MANAGEMENT STYLE ADOPTED BY THE TEAM

Our Team will be adopting the Agile Project Management Styles for this project.

11.5 INITIAL PROJECT MANAGEMENT ROLES

Mitchell Owen: Electrical Engineer, Tester

Brodie Bates: Embedded Engineer, Designer

Om Shukla: Backend Engineering

Daniel Vergara Pinilla: Machine Learning Engineer

Sudipta Halder: Simulation Developer

11.6 Team Contract

Team Name : sddec25-16

TEAM MEMBERS:

- | | |
|--------------------------|-----------------------------------|
| 1) <u>Sudipta Halder</u> | 2) <u>Daniel Mauricio Pinilla</u> |
| 3) <u>Mitchell Owen</u> | 4) <u>Brodie Bates</u> |
| 5) <u>Om Shukla</u> | |

Team Procedures

1. Day, time, and location (face-to-face or virtual) for regular team meetings:
*Saturdays from 11 AM to 2 PM can be changed based on the week.
2. Preferred method of communication updates, reminders, issues, and scheduling (e.g., e-mail, phone, app, face-to-face):
Discord and Email
3. Decision-making policy (e.g., consensus, majority vote):
majority vote
4. Procedures for record keeping (i.e., who will keep meeting minutes, how will minutes be shared/archived):
Brodie will be keeping meeting minutes. Written per meeting in Obsidian.

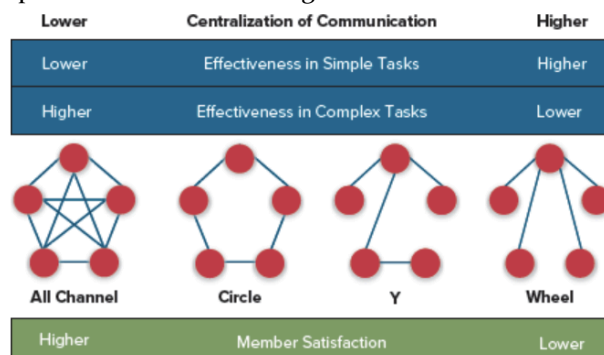
Participation Expectations

1. Expected individual attendance, punctuality, and participation at all team meetings:
Team members are expected to attend every meeting unless they are sick or for another reason cannot make it. If we agree to it, meeting online over Discord is acceptable.
2. Expected level of responsibility for fulfilling team assignments, timelines, and deadlines:
Per meeting, we will assign tasks to each member of the team or they will let the team know what they would like to work on. Those members are responsible for their section of the work regarding deadlines.
3. Expected level of communication with other team members:
We expect updates from each team member per meeting that give us an idea of what they have been working on, and also actively check or have notifications for our team on Discord.
4. Expected level of commitment to team decisions and tasks:

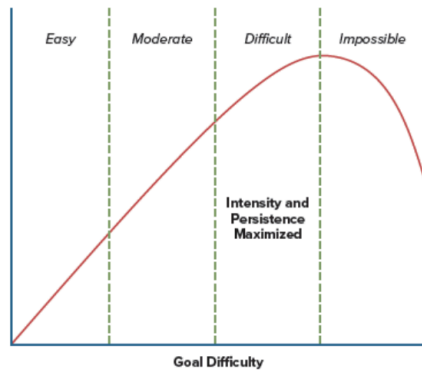
If we put something to a vote it should be voted for by all team members. This will be how we make decisions. Each member should display their opinion on team decisions if they have any, and listen to other's options, and each person has the same weight on their suggestion and none should be overlooked or ignored. All team members are expected to volunteer and pay for their tasks.

Leadership

1. Leadership roles for each team member (e.g., team organization, client interaction, individual component design, testing, etc.):
Sudipta - team organization
2. Strategies for supporting and guiding the work of all team members:
Depending on the task complexity we will use this model from the book "*Organizational Behavior: Improving Performance and Commitment in the Workplace*" by Jason Colquitt" below to increase the efficacy of the progress in our team and I might set up GitHub issues to manage tasks.



To help people to be motivated I will make sure the task is changing enough and within our time constraint as seen below which is also from the book.



3. Strategies for recognizing the contributions of all team members:
We see what people have done from GitHub commits, issues completed, and hours put in to measure how much work they have done. Hours spent divided by the time available that week will be one of the most important factors in recognizing the contributions of all team members. But since all of the team members have different schedules Another factor will be Task Complete which holds the same value as hours spent.

Collaboration and Inclusion

1. Describe the skills, expertise, and unique perspectives each team member brings to the team.

Brodie - Computer E major, programming and building experience.

Sudipta - CPRE major, programming, CAD, planning, Video making

Daniel - CPRE major, programming, full stack development, database management

Mitchell - Electrical Engineering (EE) major, CAD, and Programming: C.

Om - SE major, Programming Experience: Java, C, C++
2. Strategies for encouraging and supporting contributions and ideas from all team members:
 - We will try to manage this team with the advice from the book from the Management class one of the members took “Organizational Behavior: Improving Performance and Commitment in the Workplace”
 - Be open to any kind of suggestion.
 - Allowing anonymous suggestions so people are not afraid to share ideas.
 - Have a good work culture and a safe space. So people are more open to sharing ideas and motivated to contribute.

3. Procedures for identifying and resolving collaboration or inclusion issues (e.g., how will a team member inform the team that the team environment is obstructing their opportunity or ability to contribute?)

As soon as possible use Discord messages to inform the team or by letting certain team members know privately about the obstruction.

Goal-Setting, Planning, and Execution

1. Team goals for this semester:
Getting a working prototype where it's able to pick up trash and communicate with the server about its GPS location.
2. Strategies for planning and assigning individual and teamwork:
GitHub issues are ways we can assign individuals and teamwork, and depending on how complicated it is more collaboration will be needed. We will be using "when is good" to figure out collaboration time.
3. Strategies for keeping on task:
We will have to communicate a lot. IF STUCK REACH OUT TO THE TEAM. Do Sprint Planning (from COMS 3090) so we know what to do.

Consequences for Not Adhering to Team Contract

1. How will you handle infractions of any of the obligations of this team contract?
We will start with a simple meeting to let them know as a 1st warning.
First, we will figure out what the problem is and how we can solve it. If we are having trouble we will ask for help from elsewhere (Instructor, Advisor, organization like the student success center, etc)
2. What will your team do if the infractions continue?
2nd warning, try to resolve the situation and try something new, and if the rest of our team fails to help only then we get instructors involved.

- a) I participated in formulating the standards, roles, and procedures as stated in this contract.
- b) I understand that I am obligated to abide by these terms and conditions.
- c) I understand that if I do not abide by these terms and conditions, I will suffer the consequences as stated in this contract.

1)	<u>Sudipta Halder</u>	DATE	<u>Feb 10 2025</u>
2)	<u>Mitchell Owen</u>	DATE	<u>Feb 10 2025</u>
3)	<u>Brodie Bates</u>	DATE	<u>Feb 10 2025</u>
4)	<u>Om Shukla</u>	DATE	<u>Feb 11 2025</u>
5)	<u>Daniel Pinilla</u>	DATE	<u>Feb 11 2025</u>